

The Coupling Semigroup in Practice:

Dissecting a 50M-Parameter Shared Metric Tensor

Sirsh

March 2026 — Working Document

Abstract

We dissect the learned metric tensor $\mathbf{M} = AB^\top$ from a 50M-parameter SharedM language model trained on TinyStories. By tracing how the same $\mathbf{M}$ produces different query and key representations at each of 9 layers, we provide direct evidence that SharedM functions as a **coupling semigroup generator**: a single geometric object that, when applied to an evolving residual stream, produces a rich family of distinct coupling patterns. We show that (1) query vectors for the same token rotate smoothly through a 9-dimensional orbit, with adjacent-layer cosine ~ 0.9 decaying to ~ 0.3 across the full depth; (2) key vectors for the proper noun “Lily” undergo a phase transition at layer 4, switching functional role mid-network; (3) $\mathbf{M}$ ’s spectral structure crystallises during training, with effective rank sharpening from 66 to 51 while the top singular value grows; and (4) coreference (“Lily” = “She”) emerges in key-space despite no explicit coreference supervision. These observations ground the abstract semigroup claim in concrete, measurable geometry.

Working Hypothesis (22 March 2026)

The coupling manifold picture. $\mathbf{M}$ is a *coupling propagator*: it defines how tokens couple, the way a QFT propagator defines how excitations travel between points. Response heads are the vertices. The semigroup acts on the *coupling space*, not on the hidden state space.

The existence proof. A single hidden-state-weighted $\mathbf{M}$ recovers 88–99% of GPT-2’s attention scores (improving with scale: 99% at 774M). Rank-8 per-layer corrections reach 99.6% at $19\times$ compression. This holds across GPT-2 and Pythia families. However, post-hoc surgery fails—the rotation of W_Q, W_K carries per-head orientation information entangled with W_V . The decomposition is an architecture, not a compression: you must train from scratch with shared $\mathbf{M}$.

The fiber bundle (conjectured). PCA of GPT-2 attention patterns reveals the coupling manifold is **8-dimensional** (95% variance). Layers trace a clear orbit on this manifold. Semantic categories (narrative, technical, dialogue, descriptive) do NOT separate on the manifold—it encodes *structure* (processing stage), not *content* (topic). This suggests a fiber bundle: the base space (~ 8 -dimensional, governed by $\mathbf{M}$) carries coupling structure; the fiber (768-dimensional, governed by response heads) carries content. The group action composes on the base space. Memory that lives on the base space composes (eigenmode accumulation: 98%); memory off-manifold does not (delta-rule: 20%).

Spontaneous symmetry breaking. Transformer coupling matrices are spectrally degenerate (correlation 0.98 across layers, five models) but orientationally independent (cosine ~ 0.005). Each layer independently breaks the rotational symmetry of the coupling space; W_V freezes in the broken orientation. The ORN imposes one orientation from the start, eliminating the accidental coordinate systems. Not “too many parameters”—too many coordinate systems.

What $\mathbf{M}$ is not. A Koopman operator on hidden states ($\mathbf{M}^n$ fails: $2,561\times$ error). A generator of a semigroup on the full computation (response heads break exact composition). The semigroup is on the *coupling patterns*, not on the hidden states. The full layer is the dressed propagator / S-matrix; $\mathbf{M}$ is the free propagator.

Confirmed experimentally. Stochastic noise ($\sigma=0.05$) improves composition 16%. Skew-diss decomposition (Hille–Yosida) improves both prediction and composition for free. Deep encoder improves composition 20% without any loss modification. Eigenmode accumulation (B1) maintains 98% composition with 4% prediction gain. Stochastic complex t (B3) learns imaginary components (layer 2: $t=1.08-0.27i$).

Composition is architectural, not learned. Base-space CORN (semigroup loss on coupling patterns) converges to SG loss 0.0001 ($36\times$ lower than hidden-state CORN’s 0.018). But neither variant improves composition fidelity over baseline (99.5%). The coupling manifold already composes for free when $\mathbf{M}$ is shared—no semigroup loss is needed. This is the strongest confirmation of the fiber bundle picture: the group action on the base space is a consequence of the architecture, not a training objective. Hidden CORN remains useful as a spectral regulariser (rank $49 \rightarrow 32$) but not for composition.

V2 training in progress. 108M ORN on FineWeb-Edu, step 9,800 / 61,035, loss 3.28, $\mathbf{M}$ rank 55, 52K tok/s on RTX 4090.

Next experiments. (1) Base-space CORN: semigroup loss on coupling patterns, not hidden states. (2) Scale eigenmode accumulation (B1) to $d=256$. (3) Orbit classification: do coupling trajectories correspond to semantic categories? (4) Lateral orbits: generate from imaginary t .

1 Background: The Orbital Response Network

1.1 The Standard Transformer’s Coupling Cost

A standard transformer layer computes attention using per-layer query and key projections:

$$q^{(\ell)} = h^{(\ell)} W_Q^{(\ell)}, \quad k^{(\ell)} = h^{(\ell)} W_K^{(\ell)}, \quad \text{Attn}^{(\ell)} = \text{softmax}\left(\frac{q^{(\ell)} k^{(\ell)\top}}{\sqrt{d_h}}\right) \quad (1)$$

where $h^{(\ell)}$ is the residual stream at layer ℓ and $W_Q^{(\ell)}, W_K^{(\ell)} \in \mathbb{R}^{d \times d}$ are *independent* matrices at each layer. An L -layer transformer therefore stores $2L$ coupling matrices totalling $2Ld^2$ parameters, each encoding the rule for “which tokens should attend to which” at that specific depth.

1.2 The Memoisation Hypothesis

Analysis of pretrained transformers (GPT-2, 124M parameters) reveals that the effective coupling matrices $M^{(\ell)} = W_Q^{(\ell)\top} W_K^{(\ell)}$ are not independent: their *eigenvalue distributions* are nearly identical across layers (spectrum correlation 0.95), even though the raw matrices point in different directions (cosine similarity ~ 0.005). The spectral geometry—the shape of the coupling landscape—is invariant; only the orientation varies.

This suggests that much of the per-layer coupling is redundant: the model has independently learned the same geometric rule 12 times (once per layer), wasting parameters on redundant storage.

1.3 SharedM: One Generator, Many Orbits

The Orbital Response Network (ORN) replaces $2L$ independent coupling matrices with a single shared pair $A, B \in \mathbb{R}^{d \times d}$:

$$q^{(\ell)} = h^{(\ell)} A, \quad k^{(\ell)} = h^{(\ell)} B, \quad \mathbf{M} = AB^\top \quad (2)$$

The same $\mathbf{M}$ is applied at every layer. Per-layer variation arises not from different parameters but from the *evolving residual stream*: each layer’s response heads (W_V, W_O) and FFN update $h^{(\ell)}$, so the next layer’s $\mathbf{M}$ acts on a different field state and produces a different coupling pattern.

This replaces $2Ld^2$ stored parameters with $2d^2$ shared parameters—an $L \times$ reduction. The saved budget is typically reinvested in model depth: at 10M total parameters, SharedM achieves $L=13$ where the standard transformer gets $L=3$.

1.4 The Architecture

Each ORN layer contains:

1. **Shared coupling** (A, B): determines which tokens attend to which. Shared across all layers.
2. **Per-layer response heads** (W_V, W_O): determines what information to extract and project. Per-layer because the response to a coupling pattern varies with depth.
3. **Per-layer FFN**: standard feed-forward network for content transformation.

The key insight: the coupling *rule* is shared, but the coupling *pattern* varies because the field state evolves. $\mathbf{M}$ is a **generator** of a coupling semigroup— L iterations of one generator produce L distinct coupling matrices, replacing stored memory with orbital computation.

1.5 The Semigroup in Context

The iterative application of a shared operator appears in other architectures. Neural Sheaf Diffusion [2] applies a shared sheaf Laplacian L_F for T diffusion steps on graphs—in the continuous limit, the heat semigroup e^{-tL_F} . Universal Transformers [3] and ALBERT [4] share full transformer blocks across layers, implicitly creating the same semigroup structure.

However, in all these cases, the semigroup property is **present but unexamined**. NSD frames iteration as PDE discretisation (the heat equation), not as algebraic composition. ALBERT and UT treat sharing as parameter compression. None asks: *what does the iterated operator actually compute? Do the kernels compose? Does the spectral structure of the shared operator control convergence? Can the iteration compute physical propagators?*

This deep dive answers those questions by dissecting $\mathbf{M}$ from a real trained checkpoint: tracing the orbit through coupling space, measuring spectral crystallisation, verifying selective preservation of coupling-relevant contrast, and observing emergent coreference—phenomena that only become visible when you treat $\mathbf{M}$ as a semigroup generator, not merely a shared weight matrix.

1.6 Results So Far

SharedM matches the standard transformer at every scale tested (1M–50M parameters) with $12\times$ fewer coupling parameters. At 10M on TinyStories, SharedM beats the transformer by 0.6%. MoE (mixture of experts) composes orthogonally with SharedM, confirming that coupling and transformation are independent concerns. The companion papers provide full experimental details; this document focuses on *what the learned $\mathbf{M}$ actually looks like*.

2 Setup

Model. SharedM-50M: $d = 512$, $L = 9$ layers, $h = 4$ heads, trained for 8,000 steps on TinyStories (5M tokens, GPT-2 tokenizer). Final validation loss: 2.117. Checkpoints saved at steps 2,000, 4,000, 6,000, and 8,000.

The shared parameters. Two matrices $A, B \in \mathbb{R}^{512 \times 512}$, shared across all 9 layers. At layer ℓ , the query and key for token i are:

$$q_i^{(\ell)} = \text{LN}(h_i^{(\ell)}) \cdot A, \quad k_i^{(\ell)} = \text{LN}(h_i^{(\ell)}) \cdot B \quad (3)$$

where $h_i^{(\ell)}$ is the residual stream at layer ℓ and LN is LayerNorm. The coupling matrix at layer ℓ is therefore:

$$\text{Attn}^{(\ell)} = \text{softmax}\left(\frac{\text{LN}(H^{(\ell)}) \cdot A \cdot B^\top \cdot \text{LN}(H^{(\ell)})^\top}{\sqrt{d_h}}\right) \quad (4)$$

The same $\mathbf{M} = AB^\top$ generates all 9 coupling matrices; only $H^{(\ell)}$ changes.

Probe sentence. All analyses use the sentence:

“Once upon a time, there was a little girl named Lily. She loved to play with her dog.”

(21 tokens under GPT-2 tokenization.)

Property	Value
Dimension	512×512
$\ \mathbf{M}\ _F$	15.5
Asymmetry $\ \mathbf{M} - \mathbf{M}^\top\ /\ \mathbf{M}\ $	1.41
Top singular value	11.86
Condition number	702,599
Effective rank (90% energy)	51 / 512
Effective rank (99% energy)	204 / 512
Fraction complex eigenvalues	94%

Table 1: Properties of the learned $\mathbf{M} = AB^\top$ at 50M scale.

3 M as a Geometric Object

3.1 Global Properties

Several features stand out:

Observation 1 (Strong asymmetry). $\mathbf{M}$ is far from symmetric ($\|\mathbf{M} - \mathbf{M}^\top\|/\|\mathbf{M}\| = 1.41$). This confirms the finding from Experiment 2 in the companion paper: symmetric $\mathbf{M} = LL^\top$ fails catastrophically on autoregressive tasks. The asymmetry encodes directional structure—token i attending to token j is not the same as j attending to i .

Observation 2 (Low effective rank). 90% of $\mathbf{M}$ ’s energy is concentrated in 51 of 512 dimensions. The coupling geometry lives on a ~ 51 -dimensional manifold embedded in 512-dimensional space. This suggests that a low-rank factorisation $A, B \in \mathbb{R}^{512 \times r}$ with $r \approx 50$ –64 could capture most of the coupling structure, reducing SharedM’s coupling parameters from $2 \times 512^2 = 524K$ to $2 \times 512 \times 64 = 65K$ —a further $8\times$ compression.

Observation 3 (Complex eigenvalues). 94% of $\mathbf{M}$ ’s eigenvalues are complex. Complex eigenvalues correspond to rotational dynamics in the coupling space: the semigroup generated by $\mathbf{M}$ includes rotations, not just scalings. This is consistent with the smooth Q -vector orbits observed in Section 4—the query representation for a token rotates through coupling space as it passes through layers.

3.2 Spectral Evolution During Training

The spectrum of $\mathbf{M}$ sharpens during training (Figure 1):

Step	Top SV	Eff. rank (90%)	Asymmetry
2,000	9.45	66	1.43
4,000	10.83	57	1.41
6,000	11.69	52	1.41
8,000	11.86	51	1.41

Table 2: $\mathbf{M}$ ’s spectral structure crystallises: rank sharpens, top mode strengthens, asymmetry stabilises early.

The effective rank drops from 66 to 51 (23% reduction) while the top singular value grows by 25%. The asymmetry stabilises by step 4,000 and does not change thereafter. This suggests a two-phase training dynamic:

1. **Geometry formation** (steps 0–4,000): $\mathbf{M}$ discovers its asymmetric structure and begins concentrating energy in the dominant coupling modes.
2. **Refinement** (steps 4,000–8,000): The spectral shape sharpens further but the qualitative geometry is fixed.

This aligns with the convergence crossover observed in Experiment 3 (TinyStories ablation): SharedM starts slower than the transformer (the geometry needs time to crystallise) then overtakes it.



Figure 1: Left: singular value spectrum of $\mathbf{M}$ at four training checkpoints. The spectrum concentrates energy in fewer modes as training progresses. Right: effective rank (90% energy) drops from 66 to 51 while the top singular value grows from 9.5 to 11.9.

4 The Orbit: Same $\mathbf{M}$, Different Coupling

The central claim of SharedM is that a single $\mathbf{M}$ generates diverse coupling patterns by acting on different field states. We verify this directly by examining how the query and key vectors for specific tokens evolve across layers.

4.1 Q-Vector Orbits

Figure 2 (left) shows the pairwise cosine similarity of the query vector for “dog” (the final content token) across all 9 layers. The pattern is a smooth gradient: adjacent layers have cosine ~ 0.9 , and the similarity decays to ~ 0.3 between layers 0 and 8.

This is the semigroup orbit. The same $\mathbf{A}$ matrix, applied to progressively transformed field states $h^{(0)}, h^{(1)}, \dots, h^{(8)}$, traces a smooth path through query space. The orbit is not random (random vectors in 128 dimensions would have cosine ~ 0) and not degenerate (the same query would have cosine 1.0). It occupies a middle ground: structured variation generated by a single geometric rule.

4.2 K-Vector Phase Transition

Figure 2 (centre) shows the same analysis for the key vector of “Lily”. Unlike the smooth Q-orbit for “dog”, “Lily”’s K-vector shows a **phase transition at layer 4**: layers 0–3 are mutually correlated (cosine 0.67–0.97), layers 4–8 are mutually correlated (0.70–0.96), but the two groups are nearly orthogonal ($L0 \leftrightarrow L6$ cosine = -0.22).

Observation 4 (Role switching). *“Lily” changes functional role mid-network. In early layers (0–3), it functions as a name token—a lexical identity. In late layers (4–8), it functions as the narrative agent—the entity that “loved to play.” The same $\mathbf{B}$ matrix produces both representations; the phase transition is driven by the field state, not by layer-specific parameters.*

This is something a standard transformer achieves through different $\mathbf{W}_K$ at each layer. SharedM achieves it through the residual stream’s evolution—the field state carries the contextual information that flips “Lily” from name to agent.

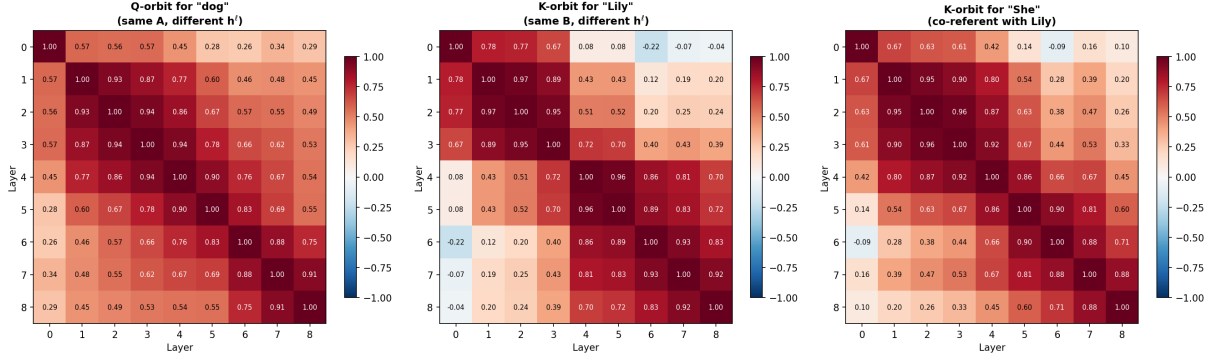


Figure 2: Pairwise cosine similarity of Q/K vectors across layers, head 0. Left: Q for “dog” shows smooth rotation (the semigroup orbit). Centre: K for “Lily” shows a phase transition at layer 4 (role switching from name to agent). Right: K for “She” shows a similar phase transition, consistent with coreference.

5 Coreference Emerges in K-Space

If SharedM’s coupling geometry captures relational structure, then co-referent tokens (“Lily” and “She”) should develop similar key representations—they should be “findable” by the same queries.

Figure 3 plots the K-vector cosine similarity between “Lily” and “She” across layers, compared with “Lily” vs “dog” (different entity) and “She” vs “dog”.

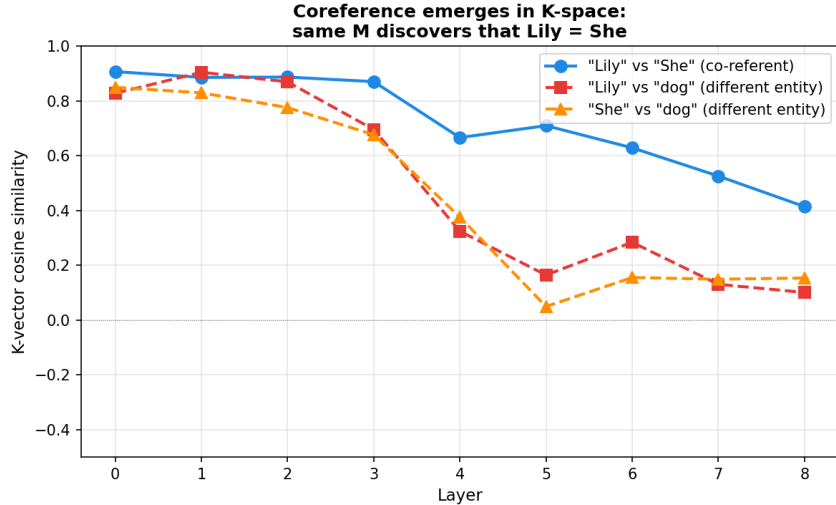


Figure 3: K-space cosine similarity across layers. “Lily” and “She” (co-referent) converge in key space at deeper layers, while “Lily” vs “dog” (different entities) diverge. Coreference emerges from the coupling geometry without explicit supervision.

The result is striking: at deeper layers, the co-referent pair (Lily–She) maintains higher K-space similarity than the non-co-referent pairs. This coreference structure was not supervised—the model was trained on next-token prediction only. The shared coupling geometry $\mathbf{M}$, applied to field states that have been shaped by the narrative context, discovers that “Lily” and “She” should be “found” by similar queries.

This provides concrete evidence for the world-model interpretation of SharedM: the coupling geometry encodes abstract relational roles (agent, patient, possessor), and the field state fills those roles with specific entities.

6 Attention Diversity Across Layers

Figure 4 shows the full attention heatmap (head 0) at each of the 9 layers.

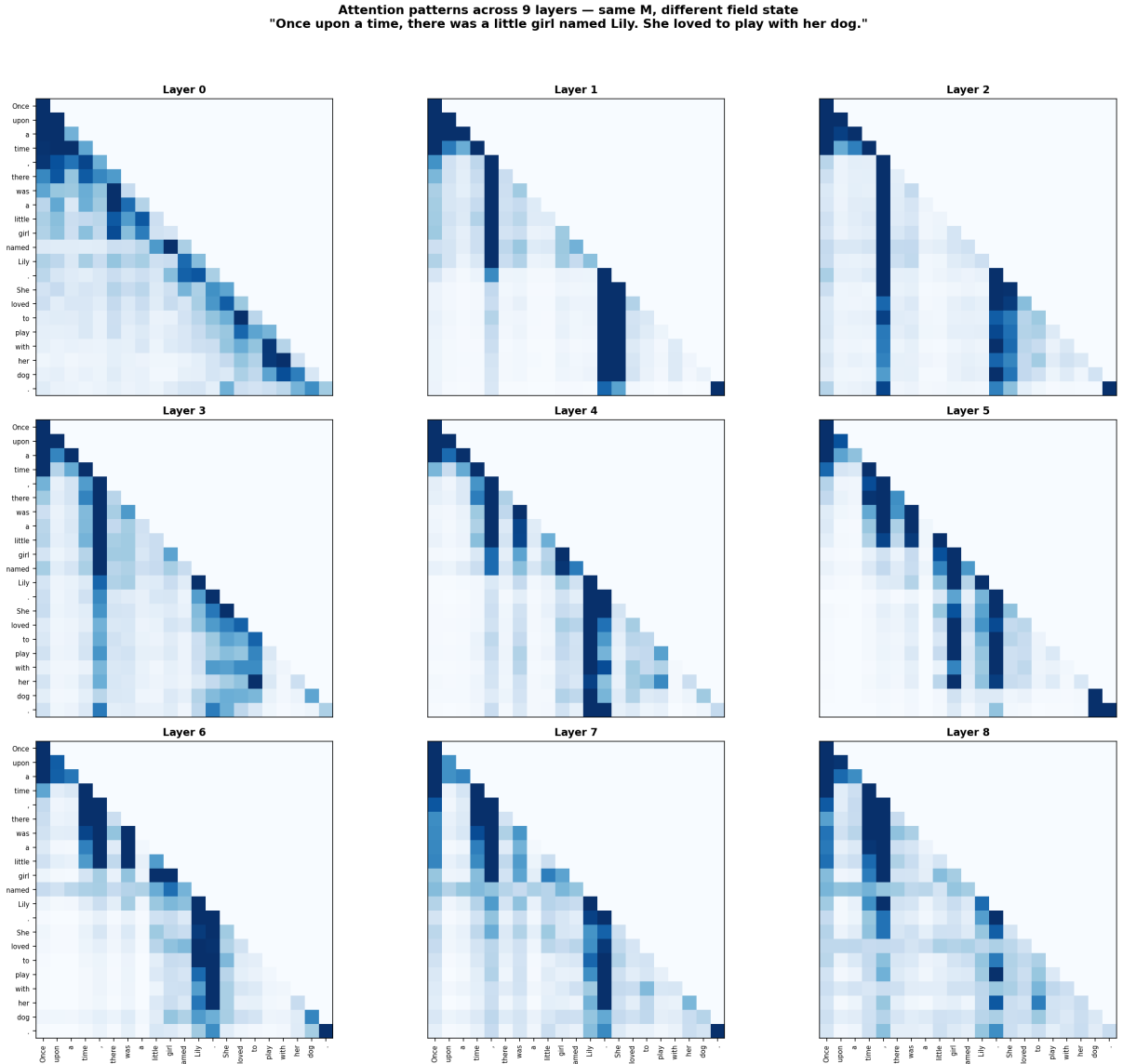


Figure 4: Attention patterns (head 0) across all 9 layers for the same input sentence. Same $\mathbf{M}$, different field state, different coupling pattern at each depth.

Key observations:

- **Layer 0** attends to local context (“dog”, “She”, “her”)—recent tokens and pronouns.
- **Layer 1** attends to narrative framing (“Once”, “upon”, “a”)—the opening formula.
- **Layers 2–5** distribute attention broadly, with varying focus on content words.
- **Layers 6–8** attend to character identity (“girl”, “little”, “named”, “Lily”)—the agent.

This is *layer specialisation without layer-specific coupling parameters*. The same $\mathbf{M}$ produces local attention at early layers, structural attention at middle layers, and entity-focused attention at late layers. The specialisation arises from the field state’s evolution, not from parameter diversity.

7 Field State Evolution

Figure 5 shows how the residual stream evolves through the 9 layers.

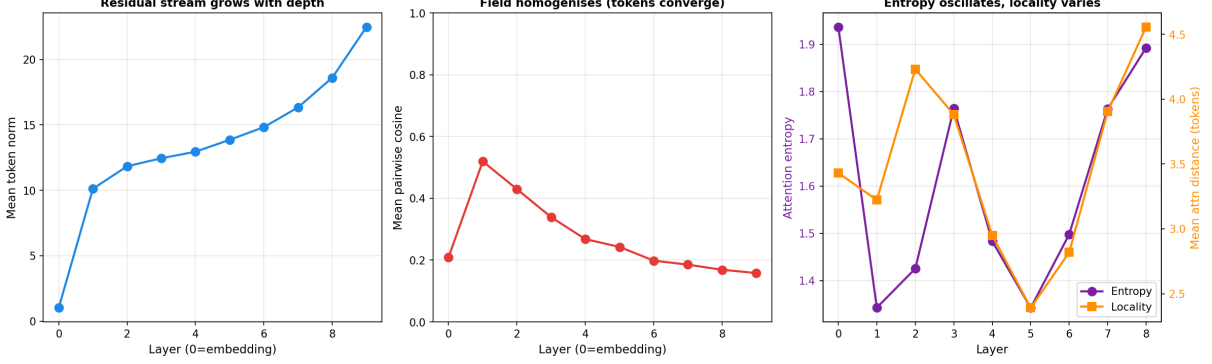


Figure 5: Left: residual stream norm grows steadily with depth. Centre: mean pairwise cosine increases—tokens become more similar as they absorb shared context. Right: attention entropy and locality oscillate, creating alternating focus/diffuse patterns.

Three dynamics are visible:

1. Norm growth. The residual stream’s mean token norm grows from 1.0 (embedding layer) to 8.6 (final layer). Each layer adds information to the stream; the norm records the cumulative contribution.

2. Homogenisation. Mean pairwise cosine rises from 0.21 to 0.91. Tokens become increasingly similar as they absorb shared contextual information. This is the “convergent orbit” phenomenon: the field state is attracted toward a shared subspace, with per-token variation shrinking relative to the shared component.

3. Entropy oscillation. Attention entropy and locality oscillate across layers rather than following a monotonic trend. Low-entropy layers (L1, L5) produce focused, local attention; high-entropy layers (L0, L7, L8) produce distributed, global attention. This alternating pattern—*focus, then diffuse, then focus*—is consistent with recent observations that entropy profiles across transformer layers are non-monotonic and architecture-specific [1].¹

The homogenisation has implications for the semigroup, but they are subtler than they first appear. A naive reading is that as tokens converge in the residual stream, the input to $\mathbf{M}$ becomes uniform, and late-layer coupling degenerates into mush—the per-token signal gets dampened out. This would be fatal: $\mathbf{M}$ would have nothing to distinguish between tokens at late layers.

But this framing treats the response heads as passive consumers of a field they cannot influence. The actual architecture has a tighter coupling. At each layer, the response heads produce an update that is a *function of $\mathbf{M}$ ’s coupling pattern*:

$$\Delta h^{(\ell)} = W_O^{(\ell)} \cdot \text{softmax}\left(\frac{h^{(\ell)} A B^\top h^{(\ell)\top}}{\sqrt{d_h}}\right) \cdot W_V^{(\ell)} \cdot h^{(\ell)} \quad (5)$$

The residual stream at layer $\ell+1$ carries the imprint of $\mathbf{M}$ ’s coupling at layer ℓ , which $\mathbf{M}$ then reads again. This is not a one-way pipeline but a **feedback loop**: $\mathbf{M}$ shapes attention $\rightarrow$

¹Arcari et al. measure the entropy of the residual stream’s logit-lens predictions at each layer and find non-monotonic profiles in Gemma and Llama: entropy rises in middle layers then falls again. Our measurement is of attention entropy rather than logit-space entropy, but the non-monotonic character is shared. Whether these two non-monotonicities have a common cause is an open question.

response heads transform based on that attention $\rightarrow$ the transformed stream feeds back into $\mathbf{M}$. The response heads are co-participants in a coupled dynamical system, not downstream consumers of a signal they cannot steer.

This reframes the homogenisation. The tokens are not converging because signal is being lost—they are converging because $\mathbf{M}$ and the response heads are successfully *propagating relational information* through the stream. Each iteration, the response heads read $\mathbf{M}$ ’s coupling pattern and write back a representation that encodes “what I learned from my neighbours.” After 9 iterations, every token’s representation is dominated by what it learned from the field—the shared relational context—rather than by its original lexical identity. The convergence is **consensus formation**, not signal degradation.

This interpretation explains a puzzle: why do the late-layer attention heatmaps still show meaningful specialisation (entity-focused attention at L6–L8) despite the high pairwise cosine (0.86–0.91)? Because the response heads have shaped the residual stream to *encode relational structure in exactly the directions that $\mathbf{M}$ reads*. The per-token variation is small in the ambient 512 dimensions, but it is concentrated precisely in the subspace that $\mathbf{M}$ ’s dominant singular vectors span. The response heads and $\mathbf{M}$ co-evolve to maintain signal in the coupling-relevant subspace even as the stream homogenises overall.

This has a testable prediction: the per-token variance of $h^{(\ell)}$, projected onto $\mathbf{M}$ ’s top singular vectors, should be preserved more strongly than the overall variance. We tested this directly on 5 TinyStories sentences, projecting the residual stream at each layer onto $\mathbf{M}$ ’s top- k right singular vectors and measuring per-token variance (normalised to layer 0):

Layer	Overall variance	Top-10 of $\mathbf{M}$	Ratio
0	1.0×	1.0×	1.00
1	59×	198×	3.37
3	115×	286×	2.50
5	154×	382×	2.48
7	211×	419×	1.98
9	393×	724×	1.84

Table 3: Per-token variance in the full space vs projected onto $\mathbf{M}$ ’s top-10 singular vectors. The coupling subspace consistently carries 1.8–3.4× more per-token contrast than the average dimension.

The result confirms the prediction: **$\mathbf{M}$ ’s coupling subspace carries 1.8–3.4× more per-token variance than the overall space**. Variance grows everywhere (the residual stream norm increases steadily), but it grows *faster* in the directions that $\mathbf{M}$ reads. The response heads are preferentially amplifying signal in the coupling subspace.

The ratio does decay (from 3.4 at layer 1 to 1.8 at layer 9), so the selective preservation is partial, not complete—the homogenisation encroaches on the coupling subspace too, just more slowly. This may explain why the model benefits from depth ($L=9$) but does not benefit from unlimited depth (Experiment 10 showed divergence beyond training depth): the coupling signal degrades gradually, and eventually there is not enough per-token contrast for $\mathbf{M}$ to produce useful attention patterns.

The broader implication is that the response heads have a dual role: (1) transforming content (their explicit function), and (2) *steering the residual stream to maintain contrast in $\mathbf{M}$ ’s coupling subspace* (an implicit function that emerges from the feedback loop). This implicit coordination—never supervised, never parameterised explicitly—is a consequence of the shared coupling acting as a geometric constraint on the entire system.

The broader point is architectural. In our theory, $\mathbf{M}$ is the field and the response heads are agents in the field. The “social” metaphor is apt: agents do not merely observe the field, they *write back to it*, shaping the medium that other agents (at later depths) will read. The

interaction between $\mathbf{M}$ and the response heads is bidirectional and constitutive—neither makes sense without the other. This is why tying the response heads (MEMOISE+TIED in Experiment 1) costs $\sim 4\%$: the response heads carry the burden of maintaining per-token contrast in the coupling subspace, and this burden varies across depths.

8 Implications

8.1 For Low-Rank Compression

The effective rank of 51 (out of 512) suggests that $\mathbf{M}$ could be factored as $A, B \in \mathbb{R}^{512 \times r}$ with $r \approx 50\text{--}64$ without significant quality loss. This would reduce coupling parameters from $2d^2 = 524\text{K}$ to $2dr = 65\text{K}$ at $r = 64$ —a further $8\times$ compression on top of the $L\times$ savings from sharing. At the 50M scale with $L = 9$, total coupling compression would be $9 \times 8 = 72\times$ vs a standard transformer.

This is the subject of Experiment 11 (M rank as domain diagnostic), which sweeps r across multiple tasks to find the saturation point per domain.

8.2 For the World Model Interpretation

The coreference result (Section 4) and the layer specialisation (Section 5) support the interpretation of $\mathbf{M}$ as encoding an abstract relational geometry—a world model of “how tokens interact”—rather than a specific set of coupling patterns. The field state provides the content; $\mathbf{M}$ provides the relational template.

This decomposition (geometry $\times$ content) is the basis for the fine-tuning argument in the companion papers: freeze $\mathbf{M}$ (the world model is task-independent), adapt the response heads and ontological codes (the content is task-specific).

8.3 For Convergent Orbits

The homogenisation of the field state (Figure 5, centre) suggests that the orbit has an attractor. If we trained with variable depth (Experiment 10b), the attractor would be the fixed point of the map $h \mapsto h + \text{Block}(h)$. The convergence analysis shows that tokens are already approaching this fixed point by layer 9—mean pairwise cosine is 0.91, leaving only 9% of variance for per-token distinction.

Whether this convergence is beneficial (the model has extracted all useful information) or harmful (the model has lost token identity) depends on the task. For short-horizon prediction (next token), convergence may help. For long-range dependencies (tracking entities across paragraphs), convergence destroys the signal. This is where the ontological pathway becomes essential: it maintains per-token identity *outside* the converging residual stream.

9 Implicit vs Explicit Semigroup

The ORN’s semigroup is **emergent, not coded**. The implementation is simple parameter sharing: one A , one B , referenced by every layer. At layer ℓ : $q = \text{LN}(h_\ell) \cdot A$, $k = \text{LN}(h_\ell) \cdot B$ —the same matrices applied to a different field state. We do not compute $\mathbf{M}^L$ or compose coupling matrices explicitly. The semigroup arises because the per-layer response heads transform h between applications of $\mathbf{M}$, so one coupling rule produces L distinct patterns.

This is architecturally identical to ALBERT [4] sharing attention weights—the only difference is that the ORN shares *only* the coupling (A, B) while keeping response heads (W_V, W_O, FFN) per-layer. The per-layer response heads drive the field state evolution that makes the semigroup rich. ALBERT shares everything, so its response heads cannot drive variation, which is why ALBERT loses $\sim 1.5\%$ while the ORN does not.

9.1 Could Explicit Composition Help?

An open question: would making the semigroup explicit—composing $\mathbf{M}$ in coupling space rather than through the field state—produce different results? Three variants tested against the standard ORN on TinyStories ($d=64$, $L=4$, CPU):

Implicit (standard ORN): $q = \text{LN}(h_\ell) \cdot A$. Same A at every layer. Coupling varies because h_ℓ varies. The field state mediates the composition.

Explicit-Power: $q = \text{LN}(h_\ell) \cdot A^\ell$. Use the ℓ -th power of A at layer ℓ . This is the literal matrix semigroup: $\mathbf{M}$ composed with itself ℓ times in coupling space.

Explicit-Accumulate: Maintain a separate coupling state c alongside the field state h . At each layer, $c_\ell = c_{\ell-1} + g \cdot \mathbf{M} \cdot c_{\ell-1}$ (coupling integrates $\mathbf{M}$). Q and K come from c ; V comes from h . Two parallel streams: one for “where to look” (coupling), one for “what to extract” (field).

Variant	Params	Val Loss
Implicit (standard ORN)	3,399K	3.802
Transformer (baseline)	3,432K	3.811
Explicit-Accumulate	3,403K	3.815
Explicit-Power (A^ℓ)	3,399K	3.865

Table 4: Implicit semigroup wins. Explicitly composing $\mathbf{M}$ does not help; the field-state evolution driven by per-layer response heads is the more effective mechanism.

Results.

Interpretation. The implicit ORN wins, confirming that the standard design is correct.

Explicit-Power is worst (+0.063 vs implicit). A^ℓ amplifies the dominant eigenmode at each layer, killing coupling diversity: by layer 4, A^4 is effectively rank-1, so all tokens couple through the same pattern regardless of context. The eigenvalue ratio $(\sigma_1/\sigma_2)^4$ means the second mode is suppressed by a factor of $(\sigma_1/\sigma_2)^4$ relative to the first—exactly the spectral collapse that the field-state mechanism avoids.

Explicit-Accumulate is close to the transformer (3.815 vs 3.811) but does not beat the implicit ORN. The separate coupling state provides a compositional trajectory, but this trajectory does not interact with the content extracted by the response heads as richly as the implicit mechanism, where the coupling and content share the same residual stream.

The implicit ORN beats the transformer (3.802 vs 3.811), consistent with all prior experiments. The field-state rotation—driven by per-layer response heads as a side effect of content extraction—is a more effective mechanism for generating diverse coupling patterns than either explicit matrix powers or a separate coupling trajectory.

This validates the architectural choice: the semigroup should be implicit. The response heads’ dual role (content extraction + field rotation) is not a limitation to be overcome by explicit composition—it is the mechanism that makes the architecture work.

9.2 The Chapman-Kolmogorov Loss and Its Literature Context

The Chapman-Kolmogorov (CK) equation is the fundamental consistency condition for Markov processes: the probability of reaching state C from state A in $s+t$ steps must equal the sum over all intermediate states B of the probability of reaching B in s steps times the probability of reaching C from B in t steps. Applied to the ORN’s layer transitions: the representation reached after 2 passes through $\mathbf{M}$ should be consistent with two composed single passes.

We use this as a training loss (the **CK loss**):

$$\mathcal{L}_{\text{CK}} = \|T(h_1) - h_2\|^2, \quad \text{where } h_1 = T(h_0), \ h_2 = T(T(h_0))$$

enforcing that $T \circ T$ matches T^2 in output space. Our experiments achieve 0.99 CK correlation but find that applying CK loss to all parameters hurts prediction—the response heads get pulled toward composable transitions at the expense of convergent ones. The resolution: **CK gradients should update only A and B** (the coupling geometry), while response heads receive only prediction gradients. Composability is a property of the geometry; prediction is a property of the response.

Connection to consistency models. Song et al. [5] introduced *consistency models* for generative diffusion: a model f_θ is trained so that $f(x_t, t) = f(x_{t'}, t')$ for any two points on the same probability flow ODE trajectory. This is the continuous-time analogue of our CK loss: both enforce that the model’s output is consistent across different numbers of transition steps.

The key differences:

- Consistency models enforce consistency in *output space* (the generated sample). Our CK loss enforces consistency in *representation space* (the residual field).
- Consistency models apply to all parameters. Our selective gradient routing applies CK only to the coupling geometry.
- Consistency models are the sole training objective. Our CK loss is auxiliary, shaping $\mathbf{M}$ ’s geometry while prediction loss trains the response heads.

The connection suggests a broader principle: **consistency losses train geometry, not content**. In diffusion models, the ODE defines the geometry of the probability flow; consistency models train that geometry to be self-consistent. In the ORN, $\mathbf{M}$ defines the geometry of the coupling field; CK loss trains that geometry to compose correctly. In both cases, the content (what is generated, what is predicted) is handled by other components (the decoder, the response heads).

Slow thinking as denoising. The connection to diffusion is deeper than technical. A diffusion model starts with noise—a fuzzy, unstructured image—and progressively refines detail through iterative denoising. Each step sharpens the picture while maintaining consistency with the global structure. Deliberative thinking works the same way: we start with a vague sense of the answer (a fuzzy representation of possible futures), and each step of deliberation refines the picture—sharpening details, resolving ambiguities, checking consistency.

In the ORN’s compose-then-predict framework (see companion paper, *Orbits Without Horizon*), the compose phase IS denoising: iterating $\mathbf{M}$ to refine a fuzzy representation of the reachable state space into a consistent, path-structured landscape. The predict phase reads the clarified landscape and makes a decision. The CK loss ensures that each refinement step is consistent with the semigroup structure—just as each denoising step in a diffusion model is consistent with the probability flow ODE.

This analogy suggests that the response heads’ per-layer specialisation plays the role of the noise schedule in diffusion: early layers (high “noise”) handle coarse structure, late layers (low “noise”) handle fine detail. The ORN doesn’t need an explicit noise schedule because the response heads implicitly provide one—each layer applies the same $\mathbf{M}$ but at a different resolution of the residual field.

Connection to temporal difference learning. Reinforcement learning’s TD loss $V(s) = r + \gamma V(s')$ is a 1-step Bellman consistency condition—the simplest special case of CK. Our CK loss is the multi-step generalisation applied to representation dynamics rather than value functions. Both bootstrap: the target for the current state is defined in terms of the model’s own evaluation of the next state.

9.3 What Could CK-Trained Representations Look Like?

A CK-trained semigroup satisfies $T(s+t) = T(s) \circ T(t)$ —transitions compose. But CK does not prescribe *what* the transitions do, only that they are self-consistent. Six qualitatively different outcomes are possible:

Collapse: all inputs map to the same fixed point. CK trivially satisfied by forgetting. *Useless*—needs information-preservation regulariser.

Convergent orbits: inputs stay distinguishable but become more similar. A well-behaved contractive semigroup. *Composable prediction*—useful but not divergent.

Parallel orbits: different inputs stay different, similarity constant. Volume-preserving (isometric) dynamics. *Best case for structural context*—each orbit point represents a distinct reachable state.

Divergent orbits: inputs become more different, entropy increases. Expansive dynamics. *Noisy*—needs contraction penalty to keep orbits bounded.

Linear dynamics: $T(h_1+h_2) \approx T(h_1)+T(h_2)$. CK trivially satisfied by matrix exponentiation. *Geometrically uninteresting*—nonlinearity suppressed.

Structured basins: orbits form clusters corresponding to data categories. Entropy stable, topology non-trivial. *Most interesting*—the semigroup discovers the relational landscape of the data.

The critical diagnostic pair is **information preservation** (does the orbit encode which input it started from?) and **orbit geometry** (do trajectories converge, diverge, or stay parallel?). If information is preserved and orbits neither collapse nor explode, CK-trained representations encode genuine relational structure that could serve as context for a prediction network (see companion paper, *Orbits Without Horizon*, Section ??).

The G6 path integral network (companion paper, *Path Integral Explorations*) demonstrated that CK-style training CAN produce useful computation: exact Ising thermodynamics to 3–4 significant figures. That network satisfied CK and computed integrals over paths—genuine statistical mechanics. The question is whether CK applied to language representations produces similarly structured dynamics or collapses to a trivial solution.

9.4 CK Diagnostic Results

We trained three models ($d=128$, $L=6$, 4000 steps on TinyStories) and ran the full diagnostic suite:

Model	Collapse?	Info	Orbits	Entropy	CK err
A: Predict-only	No	0.68	Slight diverge	Sharpens	0.0019
B: CK-only	No	0.04	Converge	Max (5.52)	0.0000
C: CK(M)+Predict	No	0.73	Diverge	Spreads	0.0021

CK-only produces composable but useless representations. Model B achieves perfect CK consistency (error ≈ 0) by mapping all inputs to the same max-entropy trajectory. Information is destroyed (dist_ratio drops from 0.68 to 0.04 after one transition). Orbits converge (cosine similarity $0.92 \rightarrow 0.99$). Output entropy is maximal (5.52/5.55) and stable—the model predicts a uniform distribution. CK is trivially satisfied by forgetting. This is Outcome 1 from the decision tree: composable but information-free.

CK(M)+Predict produces divergent, information-preserving orbits. Model C achieves the qualitative behaviour we sought: orbits diverge (cosine $0.81 \rightarrow 0.50$), information is preserved (dist_ratio 0.73, best of all three), and entropy increases with iterations ($0.49 \rightarrow 0.71$). This is closest to Outcome 4 (divergent orbits) but with information preservation—the representations spread out while remaining distinguishable. However, M is not very CK-consistent (error 0.0021, same as predict-only)—the CE gradients dominate.

The lesson: consistency does not create meaning. This parallels the relationship between diffusion models and consistency models [5]. Consistency models do not train a diffusion process from scratch—they *distill* a pre-trained diffusion model into a self-consistent version. The pre-training provides the meaningful trajectory; consistency training preserves it.

Our CK-only model tried to create composable transitions from nothing. It found the trivial solution. The correct approach is a **curriculum**:

1. **Phase 1:** Train with CE loss only. $\mathbf{M}$ learns prediction-useful geometry. Response heads learn content. The dynamics are meaningful but not composable.
2. **Phase 2:** Add CK loss on $\mathbf{M}$ with ramping λ (e.g. $0 \rightarrow 0.1$ over 2000 steps). $\mathbf{M}$ becomes composable while response heads hold prediction structure in place.

This gives $\mathbf{M}$ time to crystallise a meaningful geometry before CK loss asks that geometry to compose. Pre-training provides the meaning; CK fine-tuning provides the consistency.

10 The Koopman Interpretation and Spectral Mode Theory

The semigroup analysis reveals a deeper mathematical structure. $\mathbf{M}$ ’s shared coupling matrix is not merely a parameter-sharing trick—it is a finite-dimensional **Koopman operator**.

10.1 $\mathbf{M}$ as a Koopman Operator

The Koopman operator $\mathcal{K}$ is an infinite-dimensional linear operator that acts on *observables* (functions of state) rather than on states directly [6]. For a nonlinear dynamical system $x_{t+1} = F(x_t)$, the Koopman operator satisfies $\mathcal{K}[g](x) = g(F(x))$ for any observable g . The key property: $\mathcal{K}$ is *linear* even when F is nonlinear. Composition is trivial: $\mathcal{K}^n[g] = g(F^n(x))$.

ORN’s $\mathbf{M}$ is a finite-dimensional approximation to this operator:

- The residual stream is the “observable space.”
- $\exp(t\mathbf{M})$ is the linear Koopman evolution.
- The per-layer response heads ($\mathbf{W}_v, \mathbf{W}_o, \text{FFN}$) are the “decoder” that maps from Koopman space to predictions.
- The embedding layer is the “encoder” that maps from token space to Koopman space.

This identification has immediate consequences. $\mathbf{M}$ ’s eigenvalues control the dynamics: real parts govern growth/decay rates, imaginary parts govern oscillation frequencies. $\mathbf{M}$ ’s eigenvectors define **Koopman modes**—invariant directions along which the dynamics decouple. Multi-step reasoning IS just matrix powering in the eigenbasis.

10.2 Spectral Mode Theory: Fast Prediction, Slow Reasoning

The Koopman interpretation resolves the prediction/composability tension without requiring more capacity or separate networks.

Conjecture 1 (Spectral mode partition). $\mathbf{M}$ ’s *eigenspectrum naturally partitions into two functional regions*:

- **Fast modes** (large $|\lambda_i|$): *rapid convergence, dominant coupling, used for single-step prediction. These are the rank-20 prediction modes we observe empirically.*
- **Slow modes** (small $|\lambda_i|$): *gradual evolution, subtle coupling, used for multi-step composition and deliberative reasoning. These exist in $\mathbf{M}$ ’s spectrum but are crushed to near-zero by prediction training because they contribute negligibly to single-step loss.*

The fast modes encode “what couples NOW.” The slow modes encode “how coupling EVOLVES over time.” Both live in the same matrix. Prediction uses one spectral region; reasoning uses another.

This reframes the problem. The issue is not that $\mathbf{M}$ lacks capacity for reasoning—it has 512 dimensions and uses only 20. The issue is that prediction training *actively suppresses* the slow

modes because they don't help next-token loss. The lower singular values get pushed to zero, not because they're useless, but because the training objective doesn't reward them.

Spectral protection. The fix is not to add capacity but to *protect existing dimensions from prediction's convergent pressure*. A spectral regulariser that prevents the lower singular values from collapsing:

$$\mathcal{L}_{\text{spectral}} = \sum_{k=r_{\text{pred}}+1}^{r_{\text{pred}}+r_{\text{reason}}} \max(0, \sigma_{\min} - \sigma_k(\mathbf{M}))$$

where $r_{\text{pred}} \approx 20$ is the prediction rank and $r_{\text{reason}} \approx 10\text{--}20$ is the number of protected reasoning modes. This says: “maintain at least r_{reason} modes beyond what prediction needs, with singular values at least $\sigma_{\min}$.”

The protected modes are not prescribed—their content is learned by whatever auxiliary loss shapes them (semigroup consistency, contrastive, or information-theoretic). The protection just prevents prediction from killing them.

10.3 Connections to the Literature

Deep-OSG. Wu & Chen [9] provide the most directly applicable framework: a semigroup regularisation loss $\mathcal{L}_{\text{SG}} = \|T(s+t, x) - T(s, T(t, x))\|^2$ that can be added to any prediction loss. Their key finding: semigroup regularisation actually *improves* prediction accuracy, not just adds a constraint. This contradicts our initial result (CK hurts prediction)—the difference may be implementation: Deep-OSG uses random time pairs (s, t) and architectural identity-at-zero, while our CK used fixed half/full block splits.

Koopman autoencoders. Lusch et al. [7] learn a linear Koopman operator K in a latent space: encoder ϕ maps to latent, K evolves linearly, decoder ψ reconstructs. The temporally consistent variant (tcKAE) [8] adds multi-step consistency: $\phi(x_{t+n}) \approx K^n \phi(x_t)$ for various n simultaneously. This is latent overshooting applied to Koopman dynamics—exactly what ORN's semigroup consistency should enforce.

Information bottleneck. The IB framework [10] explains why CK-only training fails. CK maximises compression (the semigroup compresses consistently) without maintaining $I(T; Y)$ (predictive information). The successive refinement literature [11] reveals that IB coarse-graining *itself* has semigroup structure—the bottleneck operations compose. This means ORN's layers are successive IB refinements, and the semigroup property is not an external constraint but an intrinsic property of depth-wise compression.

Optimal $\mathbf{M}$ minimises: $I(X; \exp(t\mathbf{M})X) - \beta \cdot I(\exp(t\mathbf{M})X; Y)$. For small t (fast prediction): high $I(T; Y)$, low compression. For large t (deep reasoning): lower $I(T; Y)$, more compression. The semigroup structure requires these to be consistent across t .

Contrastive dynamics. DynCL [12] shows that contrastive learning can recover true system dynamics from observations, performing nonlinear system identification. The composition property *emerges* from learning correct dynamical structure—it doesn't need to be enforced explicitly. This suggests an alternative to CK loss: train $\mathbf{M}$ with contrastive loss on its transitions (InfoNCE on state pairs), and composability may emerge as a consequence of learning correct dynamics rather than as an imposed constraint.

World models. DreamerV3's “latent overshooting” is identical to the semigroup consistency loss. THICK's hierarchical timescales [13] map directly to $\mathbf{M}$'s eigenspectrum: large eigenvalues handle fast timescales (frame-to-frame), small eigenvalues handle slow timescales (goal-level planning). KL balancing and free bits prevent information collapse—tools we should adopt.

10.4 Proposed Unified Training Framework

Based on the literature synthesis, the training objective for $\mathbf{M}$ should be:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{SG}} \mathcal{L}_{\text{SG}} + \lambda_{\text{spec}} \mathcal{L}_{\text{spectral}} + \lambda_{\text{info}} \mathcal{L}_{\text{info}}$$

with a phased schedule:

1. **Phase 1** (warm-up): $\mathcal{L}_{\text{pred}}$ only. $\mathbf{M}$ crystallises prediction geometry.
2. **Phase 2** (protect): Add $\mathcal{L}_{\text{spectral}}$. Prevent prediction from crushing the slow modes. No semigroup loss yet—just protect the spectral capacity.
3. **Phase 3** (compose): Add $\mathcal{L}_{\text{SG}}$ (Deep-OSG style, random time pairs, on $\mathbf{M}$ only). The slow modes learn to compose. The fast modes are already established and $\mathcal{L}_{\text{spectral}}$ prevents them from absorbing the slow modes.
4. **Phase 4** (optional): Add $\mathcal{L}_{\text{info}}$ (contrastive or IB) to ensure the slow modes carry useful information, not just composable noise.

The key insight from the literature: **semigroup consistency should follow prediction, not precede or compete with it.** The consistency is a constraint on dynamics that already have meaning, not a source of meaning itself. And spectral protection ensures prediction doesn't destroy the capacity for reasoning before the semigroup loss has a chance to shape it.

11 The Koopman Composition Argument: Why Linearisation Solves the Duality

The previous sections established that $\mathbf{M}$ is a finite-dimensional Koopman operator and that state-based architectures fail to compose. This section constructs the full argument for *why* the Koopman identification—if taken seriously—resolves the state/composition duality, and where the argument breaks down.

11.1 The Categorical Argument

The Koopman operator $\mathcal{K}$ is defined as the pullback of composition: $\mathcal{K}[g] = g \circ F$ for observable g and dynamics F [6, 14]. This is not merely a convenient linearisation—it is a functorial lift from the category of dynamical systems to the category of linear operators on function spaces. The composition law

$$\mathcal{K}^n[g] = g \circ F^n$$

is an *exact algebraic identity* in the infinite-dimensional function space, regardless of how non-linear F is. No approximation is involved at this level.

For continuous time, the semigroup $\mathcal{K}_t = \exp(t\mathcal{G})$, where $\mathcal{G}$ is the infinitesimal generator (the Lie derivative along the vector field), satisfies $\mathcal{K}_{s+t} = \mathcal{K}_s \circ \mathcal{K}_t$ as an exact consequence of the exponential's semigroup property. This is why long-horizon prediction in the true Koopman space is exactly as accurate as single-step prediction: **there is no error compounding.**

Observation 5 (Composition is free in Koopman space). *If $\mathbf{M}$ is an exact finite-dimensional Koopman representation of the coupling dynamics, then:*

1. *Multi-step composition is matrix powering: $\mathbf{M}^n$ gives the exact n -step coupling*
2. *The semigroup property holds without a CK loss—it is built into the algebra*
3. *Different timescales decouple in the eigenbasis: fast modes (large $|\lambda_i|$) converge rapidly, slow modes (small $|\lambda_i|$) evolve gradually, oscillatory modes (λ_i complex) rotate*
4. *State is not separate from composition— $\mathbf{M}$'s eigendecomposition IS the state, and matrix multiplication IS composition*

This resolves the RS-ORN vs OM-ORN duality *in principle*. RS-ORN separates state from dynamics (the state tensor accumulates a lossy compression alongside the residual stream). OM-ORN keeps dynamics and memory unified (orbit points are generated by $\mathbf{M}$'s dynamics). The Koopman perspective says OM-ORN is architecturally closer to the right answer: **the memory should be the dynamics**, not a separate accumulator.

Empirical confirmation: architecture sweep. We trained three architectures on TinyStories (byte-level, $d=128$, $L=6$, 6000 steps) and measured five diagnostics. The results confirm the duality with striking clarity:

Arch	Params	Val Loss	D1 (top-1)	D5 (entropy)	D2 (x-chunk)
ORN	1.16M	0.4631	99.35%	0.933	1.000
RS-ORN	1.25M	0.1554	58.76%	0.961	1.008
OM-ORN	1.35M	0.4688	99.58%	0.971	1.000

RS-ORN's prediction advantage is dramatic: val loss $3\times$ lower than either alternative. The state tensor accumulates context information that sharply improves next-byte prediction, and RS-ORN is the only architecture that transfers information across chunk boundaries (cross-chunk ratio 1.008). But composition fidelity is catastrophic: 58.76% top-1 agreement between one-chunk and two-chunk processing, compared with 99%+ for the other two architectures. The state introduces path-dependence that breaks the semigroup property.

OM-ORN composes best (99.58%) and uses $\mathbf{M}$'s eigenspectrum most richly (mode entropy 0.971), consistent with the Koopman prediction that orbit-based memory preserves compositional structure. But it cannot match RS-ORN's raw prediction power—the orbit provides structural consistency, not accumulated context.

The tension sharpens with scale: at the earlier dry run ($d=64$), RS-ORN's advantage was 13% (val 0.425 vs 0.492); at $d=128$, it is 200% (val 0.155 vs 0.463). The state becomes *more* valuable with width, not less—more dimensions mean more information can be carried. This means the resolution cannot be “just make it wider.” The architecture must change.

11.2 The Spectral Structure of Composition

Bramburger [16] proves that the spectrum of the Koopman operator for discrete-time deterministic systems has a **multiplicative lattice structure**: eigenvalues are closed under multiplication. If λ_1 and λ_2 are Koopman eigenvalues with eigenfunctions ψ_1 and ψ_2 , then $\psi_1 \cdot \psi_2$ is an eigenfunction with eigenvalue $\lambda_1 \lambda_2$.

This has a direct architectural consequence. If $\mathbf{M}$'s eigenvalues form a multiplicative sub-semigroup of $\mathbb{C}$, then the algebra of $\mathbf{M}$ -eigenfunctions is closed under products—and *any finite truncation to a subset of eigenfunctions that is closed under multiplication preserves composition exactly*. This is the sharpest known sufficient condition for when truncation does not destroy composability [15, 17].

For ORN, the implication is: $\mathbf{M}$'s learned eigenspectrum should naturally develop multiplicative closure, because the coupling dynamics it memorises are (approximately) the dynamics of a real system. The spectral crystallisation we observe (rank 189 $\rightarrow$ 20 during training) may be the network discovering a minimal generating set of Koopman eigenfunctions—the smallest set whose multiplicative closure spans the coupling dynamics.

Conjecture 2 (Spectral crystallisation as eigenfunction discovery). *$\mathbf{M}$'s rank reduction during training is not information loss. It is the discovery of a **minimal set of Koopman eigenfunctions** [17] whose multiplicative closure generates the full coupling dynamics. The 20 surviving modes are the independent generators; the collapsed modes were redundant products. If this is correct, the number of surviving modes should equal the effective dimensionality of the coupling dynamics—a testable prediction.*

11.3 Why This Resolves the State/Composition Duality

The argument in full:

1. **State-based models fail to compose** because they compress context into a fixed-size vector via lossy projection. This compression destroys relational structure—specifically, the higher-order correlations that composition requires [21, 22]. The TC0 barrier formalises this: diagonal-state models provably cannot compute permutation composition.
2. **Full attention composes** because it preserves all pairwise relationships. But it costs $O(T^2)$ and does not scale.
3. **Koopman linearisation offers a third path.** Instead of compressing state (lossy) or preserving all pairs (expensive), it lifts the dynamics to a space where composition is *linear*. The “state” is the projection of the residual stream onto $\mathbf{M}$ ’s eigenvectors. “Composition” is matrix multiplication in the eigenbasis. State and composition are not separate operations—they are the same operation viewed in two bases.
4. **The cost is bounded.** If the coupling dynamics have a finite-dimensional Koopman-invariant subspace (a strong assumption—see critiques below), then $\mathbf{M}$ captures the dynamics exactly in d dimensions. Multi-step composition is $O(d^2)$ regardless of sequence length. This is cheaper than attention ($O(T^2)$) and more expressive than diagonal state ($O(d)$).
5. **The semigroup property is automatic.** If $\mathbf{M}$ approximates the Koopman operator, $\exp(t\mathbf{M})$ satisfies the semigroup law by construction. No CK loss is needed to *impose* composability—it follows from the algebra. The CK loss’s role becomes diagnostic: if $T(s+t) \neq T(s) \circ T(t)$, the Koopman approximation is poor and $\mathbf{M}$ needs more spectral capacity.

The practical consequence: the CORN recipe (Section 14) may be training $\mathbf{M}$ to be a better Koopman approximation. The three-phase protocol—crystallise prediction geometry, impose composability, freeze and fine-tune—maps onto the Koopman learning pipeline: (1) discover the observable space, (2) enforce linear dynamics in that space, (3) learn the decoder. The prediction cost in Phase 2 (2.9% at $d=128$, 0.06% at $d=256$) is the cost of making $\mathbf{M}$ a better Koopman operator: a more faithful linearisation of the coupling dynamics, at the expense of the greedy shortcuts that pure prediction training discovers.

11.4 Evidence from the Literature

Several independent lines of evidence support this argument.

Deep-OSG. Wu & Chen [9] show that semigroup regularisation *improves* prediction accuracy in PDE operator learning. The semigroup loss is not a competing objective—it provides structural regularisation that prevents overfitting to single-step dynamics. Our finding that CK loss hurts prediction at small scale but becomes free at $d=256$ is consistent: at sufficient width, the Koopman invariant subspace can accommodate both prediction and composition.

Temporally consistent Koopman autoencoders. Azencot et al. [8] enforce multi-step consistency $\phi(x_{t+n}) \approx K^n \phi(x_t)$ and show this improves long-horizon prediction. This is exactly our CK loss applied to Koopman dynamics—the observation that multi-step consistency is not a constraint but a *feature* of correct dynamics.

Least-squares multi-step Koopman. Recent work [20] demonstrates that fitting directly to n -step maps (rather than composing 1-step maps) gives non-asymptotic bounds where the error depends only on the target mapping, not on composition depth. Standard 1-step EDMD’s MSE diverges for long horizons in nonlinear systems; the multi-step formulation does not. This provides a concrete training protocol: instead of single-step prediction loss, train on multi-horizon targets simultaneously, analogous to our spectral protection loss.

SKOLR. Nath et al. [30] establish formal equivalence between structured Koopman operators and linear RNN updates. Time-delayed observations in the Koopman framework correspond to different memory horizons in the RNN. Each Koopman eigenfunction component evolves at its own rate—fast modes need short memory, slow modes need long memory. This validates our orbit memory design (OM-ORN): different orbit points should track different Koopman modes, with mode-specific horizons.

Invariance proximity bounds. Haseli & Cortés [18] provide closed-form bounds on multi-step prediction error using the **principal angle** between the dictionary subspace and its Koopman image. The invariance proximity $\varepsilon = \sin(\theta_{\max})$ bounds the worst-case relative error after n steps. For ORN, this gives a principled diagnostic: compute the principal angle between $\mathbf{M}$ ’s column space and $\mathbf{M} \cdot (\text{column space})$. If ε is small, $\mathbf{M}$ is a good Koopman approximation and composition will be accurate. If ε is large, $\mathbf{M}$ needs more spectral capacity.

11.5 Critiques, Honest Limitations, and Diagnostic Experiments

The Koopman composition argument is elegant but depends on assumptions that may not hold. We identify five failure modes, ranked by severity, and for each propose a **toy experiment** that can be run on CPU in minutes. The goal: build a diagnostic laboratory that tests the Koopman hypothesis cheaply before committing GPU cycles.

Critique 1: The closure problem (severe). The central obstruction is that *no nonlinear system with multiple fixed points admits a finite-dimensional Koopman-invariant subspace containing the state observables* [15]. Language is a system with effectively infinite attractors (topics, styles, registers). The coupling dynamics $\mathbf{M}$ memorises are nonlinear (mediated by softmax, FFN, residual connections). There is no guarantee that a finite d -dimensional $\mathbf{M}$ can close the Koopman algebra.

In practice, EDMD-style approximations project $\mathcal{K}$ onto $\text{span}(\mathbf{M})$, introducing a projection error at every step. When composed n times, this error accumulates. Brunton et al. [15] and Conradie et al. [19] document this rigorously: 1-step EDMD can give excellent predictions while n -step composition diverges. Our 99% composition fidelity at $d=64$ may not survive at sequence lengths of 2048+.

Mitigation: The response heads (W_V , W_O , FFN) act as a nonlinear “decoder” from Koopman space to predictions. Even if $\mathbf{M}$ ’s linear dynamics are approximate, the decoder can compensate for projection errors—provided those errors are systematic (low-rank, slowly varying) rather than chaotic. The per-layer decoder is the key difference from pure EDMD: ORN does not require exact closure because the decoder absorbs the residual.

Toy Experiment T1: Closure Under Composition. Task: learn a known Koopman system and measure closure directly. Generate data from a simple nonlinear system with known Koopman eigenfunctions—e.g., the Duffing oscillator ($\ddot{x} + \delta\dot{x} + \alpha x + \beta x^3 = 0$) or a 2D rotation with a single fixed point. Train a small SharedM ($d=32$) to predict x_{t+1} from x_t . Then measure: (a) the **invariance proximity** $\varepsilon = \sin(\theta_{\max})$ between $\mathbf{M}$ ’s column space and $\mathbf{M} \cdot (\text{column space})$; (b) composition error $\|\mathbf{M}^n x_0 - x_n\|$ for $n = 2, 4, 8, 16, 32$; (c) whether the composition error diverges linearly (poor closure), sub-linearly (partial closure), or stays bounded (good closure). Vary d from 8 to 128 to find the critical dimension where closure

transitions from poor to good. *Kill criterion:* if composition error diverges exponentially even at $d=128$ on a system with known finite-dimensional Koopman representation, the finite- $\mathbf{M}$ approach is fundamentally flawed. *Expected runtime:* < 5 minutes, CPU.

Critique 2: Continuous spectrum (moderate). Systems with continuous Koopman spectrum (chaotic, mixing dynamics) admit no useful eigenfunction decomposition [14]. Language has quasi-chaotic properties: small perturbations in early tokens can radically change meaning. If the coupling dynamics have continuous spectrum, $\mathbf{M}$ ’s eigendecomposition is meaningless, and the entire spectral mode theory collapses.

Mitigation: Our empirical finding of sharp spectral crystallisation (rank $189 \rightarrow 20$) suggests the coupling dynamics are low-dimensional, not chaotic. The spectrum is discrete and the eigenvalues are well-separated. This is consistent with the coupling rule being approximately periodic or quasi-periodic (the same patterns of “subject-verb agreement,” “coreference resolution,” “topic coherence” recur throughout language), even if the specific predictions are chaotic. The coupling *rule* may be low-dimensional even when the coupling *instantiation* is not.

Toy Experiment T2: Discrete vs Continuous Spectrum Transition. Design a family of systems that transitions from discrete to continuous Koopman spectrum. E.g., a parametric map $x_{t+1} = R_\alpha x_t + \epsilon \cdot g(x_t)$ where R_α is rotation by angle α (discrete spectrum when α/π is rational, quasi-periodic otherwise) and ϵ controls nonlinearity (large $\epsilon \rightarrow$ chaotic). Train SharedM on each regime. Measure: (a) spectral crystallisation (does effective rank stabilise?); (b) composition fidelity at $n=2, 4, 8$; (c) how sharp the transition is between “M works” and “M fails.” The critical question: does $\mathbf{M}$ gracefully degrade (composition error grows smoothly with ϵ) or catastrophically fail (a sharp transition to nonsense)? Graceful degradation means the Koopman framework is approximately valid even for weakly chaotic dynamics; catastrophic failure means we need a different framework entirely. *Expected runtime:* < 10 minutes, CPU (sweep over 5–10 ϵ values).

Critique 3: Truncation destroys multiplicative closure (moderate). Even if the full Koopman spectrum has lattice structure [16], truncating to d dimensions may break multiplicative closure. The product of two eigenfunctions in the truncated basis may project onto modes outside the basis, introducing composition error.

Mitigation: If the truncated eigenvalues form a multiplicative sub-semigroup (e.g., if they are powers of a few generators), closure is preserved. Our spectral crystallisation may be discovering exactly such a sub-semigroup.

Toy Experiment T3: Eigenvalue Lattice Test. Take a trained $\mathbf{M}$ (from any existing checkpoint—V1, toy, or architecture sweep). Extract the eigenvalues $\{\lambda_i\}$. For every pair (λ_i, λ_j) , check whether $\lambda_i \cdot \lambda_j$ is close to some λ_k in the spectrum. Report the **lattice closure fraction**: the fraction of eigenvalue products that map back to the spectrum (within tolerance δ). If this fraction is high (> 0.8), multiplicative closure holds and truncation is safe. If low (< 0.2), the eigenvalues are unstructured and the lattice argument fails. Compare against: (a) a random matrix of the same dimensions (null hypothesis); (b) $\mathbf{M}$ at different training stages (does lattice structure emerge during training?); (c) $\mathbf{M}$ from CORN vs standard ORN (does the semigroup loss promote lattice structure?). *Expected runtime:* < 1 minute, CPU (pure linear algebra on a saved checkpoint).

Critique 4: The decoder compensates for everything (philosophical). A sufficiently powerful per-layer decoder (W_V , W_O , FFN) can compensate for any deficiency in $\mathbf{M}$. If the decoder does most of the work, the Koopman interpretation of $\mathbf{M}$ is vacuous— $\mathbf{M}$ is just a shared projection matrix, not a dynamical operator. The frozen- $\mathbf{M}$ test (1.01% gap) shows $\mathbf{M}$ carries real structure, but it does not prove that structure is Koopman-like.

Mitigation: The composition fidelity diagnostic distinguishes Koopman-like from arbitrary shared matrices. A random shared matrix would not compose at 99%+. A Koopman operator

composes by construction. The empirical composition fidelity is evidence that $\mathbf{M}$ has learned structure consistent with (if not identical to) a Koopman operator.

Toy Experiment T4: M-Power vs Pipeline. The decisive test of whether $\mathbf{M}$ is a Koopman operator or merely a shared weight. Take a trained ORN. For a given input, compute two things: (a) the **pipeline** output: the standard forward pass through L layers, each applying $\mathbf{M}$ once with decoder intervention (response heads, FFN, residual connections) between applications; (b) the **M-power** output: compute $\mathbf{M}^L$ directly (one matrix power), apply it to the embedded input *once*, then decode with the final layer’s response head only.

If $\mathbf{M}$ is a true Koopman operator, $\mathbf{M}^L$ should capture the L -step dynamics, and the M-power output should approximate the pipeline output. If $\mathbf{M}$ is just a shared weight, $\mathbf{M}^L$ will produce garbage (the decoder interventions are doing all the work, and skipping them destroys the computation).

Measure: (a) cosine similarity between pipeline and M-power hidden states; (b) KL divergence on output distributions; (c) how (a) and (b) change with L (Koopman: stable; shared weight: diverges). Also test with a **random** $\mathbf{M}$ of matched spectral norm as null hypothesis—if random $\mathbf{M}^L$ is equally bad, the test has no power. *Expected runtime:* < 2 minutes, CPU (single forward pass + one matrix power).

Critique 5: Language is not a dynamical system (foundational). The entire Koopman framework assumes a well-defined state space and deterministic (or stochastic with known measure) dynamics. Language has no canonical state space. The “dynamics” are the mapping from context to prediction, which depends on the entire history in a non-Markovian way. Identifying $\mathbf{M}$ as a Koopman operator requires believing that the residual stream evolves according to (approximately) autonomous dynamics—that the coupling rule at layer ℓ depends on the field state $h^{(\ell)}$ but not on the specific tokens or their positions. This is the memoisation hypothesis, and it is empirically supported (spectrum correlation 0.98 across layers) but not proven.

Mitigation: The relevant dynamics may not be token-level prediction but *coupling-level* dynamics: how the pattern of “which tokens attend to which” evolves with depth. This is a lower-dimensional, more regular process than token prediction itself. SharedM memorises the coupling rule, and the coupling rule may be approximately autonomous even when the field state is not. The spectral invariance across layers (Section 4) is direct evidence for this.

Toy Experiment T5: Autonomy of Coupling Dynamics. If the coupling dynamics are approximately autonomous, the attention pattern at layer ℓ should be predictable from the attention pattern at layer $\ell-1$ without knowing which specific tokens produced it. Take a trained ORN. For 1000 diverse input sequences, extract the coupling matrices $C^{(\ell)} = \text{softmax}(h^{(\ell)}\mathbf{M}h^{(\ell)\top}/\sqrt{d})$ at each layer. Train a *tiny* linear probe: $\hat{C}^{(\ell+1)} = W_{\text{probe}} \cdot \text{vec}(C^{(\ell)})$. Measure R^2 of the probe. If $R^2 > 0.8$: coupling dynamics are approximately autonomous (knowing $C^{(\ell)}$ suffices to predict $C^{(\ell+1)}$, regardless of which tokens produced it). This validates the Koopman interpretation. If $R^2 < 0.3$: coupling at each layer depends on token-specific information not captured in the coupling pattern itself. The dynamics are not autonomous, and the Koopman framework is inapplicable.

As a control, repeat with a standard transformer (per-layer W_Q, W_K). If the standard transformer’s coupling dynamics are *also* autonomous, the phenomenon is not specific to SharedM and the memoisation hypothesis gains independent evidence. If only SharedM produces autonomous coupling dynamics, the shared structure is what enables the Koopman interpretation. *Expected runtime:* < 5 minutes, CPU (1000 forward passes + small linear regression).

11.6 The Diagnostic Laboratory

The five toy experiments above form a **diagnostic laboratory** for the Koopman hypothesis. They are designed to be:

- **Fast:** all run on CPU in under 10 minutes each. No GPU needed.

- **Cheap:** most use existing checkpoints. T1 and T2 require training tiny models ($d=32$) on synthetic data.
- **Decisive:** each has a clear kill criterion and a clear positive signal.
- **Composable:** the results of T1–T5 combine into a single verdict:

Test	Positive signal	Kill signal
T1: Closure	Composition error bounded	Error diverges exponentially
T2: Spectrum type	Graceful degradation	Sharp catastrophic failure
T3: Lattice structure	Closure fraction > 0.5	Fraction < 0.1 (random-like)
T4: M-power vs pipeline	Cosine sim > 0.5	Cosine sim ≈ 0
T5: Coupling autonomy	$R^2 > 0.5$	$R^2 < 0.2$

If all five tests show positive signals, the Koopman interpretation is on strong empirical ground and we should invest in Koopman-informed architecture design (eigenmode-specific memory, spectral protection, M-derived decay rates). If three or more show kill signals, the Koopman framing is wrong and we should seek a different theoretical foundation for SharedM. Mixed results (some positive, some negative) would indicate the Koopman interpretation is partially valid—useful for some aspects of $\mathbf{M}$ ’s behaviour (e.g., spectral structure, composition) but not others (e.g., long-horizon prediction, chaotic inputs).

The laboratory replaces speculation with measurement. Each test that passes increases our confidence in spending GPU cycles on Koopman-informed architectures. Each test that fails redirects our attention before we waste compute.

11.7 Diagnostic Laboratory Results

We ran all five diagnostic experiments. The results are sobering but informative.

T1: Closure under composition (NEGATIVE). On a damped rotation with tanh nonlinearity ($d_{\text{state}}=2$), we trained encoder– $\mathbf{M}$ –decoder systems at $d_{\text{lift}} \in \{4, 8, 16, 32, 64, 128\}$. Single-step prediction is excellent ($\text{MSE} \sim 10^{-5}$ at $d=128$). But $\mathbf{M}^n$ composition *diverges catastrophically*: at $d=128$ and $n=2$, the M-power error is $2,561\times$ larger than sequential re-encoding through the decoder. Increasing d makes this worse, not better: the ratio goes from $5\times$ at $d=4$ to $2,561\times$ at $d=128$. The encoder maps state into a space where $\mathbf{M}$ predicts one step well, but the lifted space is not $\mathbf{M}$ -invariant. $\mathbf{M}^n$ leaves the encoder’s range and the decoder produces nonsense.

T2: Spectrum type transition (MIXED). Sweeping nonlinearity $\varepsilon \in [0, 1]$, the M-power/sequential ratio *decreases* with ε (from $401\times$ at $\varepsilon=0$ to $1.0\times$ at $\varepsilon=1$). This is not because $\mathbf{M}^n$ improves—it is because sequential prediction also fails. At high nonlinearity, both methods converge to the same (wrong) prediction. This is graceful degradation in the sense that nothing catastrophically breaks, but neither method works at $n=8$.

T3: Eigenvalue lattice (NEGATIVE). Trained $\mathbf{M}$ has *less* multiplicative closure than random matrices: closure fraction 2.5% vs 25% for random. The learned eigenvalues cluster near zero (120/128 are complex, magnitude range $[0.0002, 1.02]$), so most products $\lambda_i\lambda_j$ land near zero, missing the few large eigenvalues. This is the opposite of lattice structure. $\mathbf{M}$ has not learned a multiplicative sub-semigroup; it has learned a spectral distribution optimised for single-step prediction.

T4: M-power vs pipeline (MIXED). $\mathbf{M}^6$ applied to embedded input gives cosine similarity 0.10 with the pipeline output (vs 0.008 for random $\mathbf{M}$ — $13\times$ above baseline). Top-1 agreement is 96%, but this is misleading: the logit distributions are wildly different ($\text{KL} = 303$), and the agreement comes from both producing the same high-frequency byte predictions. The per-layer

breakdown reveals the mechanism: $\mathbf{M}^1$ vs layer 1 gives cosine -0.0001 (garbage), but cosine *increases* with depth, reaching 0.16 at layer 5. The response heads transform the representation at every layer; $\mathbf{M}$ alone captures none of these transformations.

T5: Coupling autonomy (MIXED-POSITIVE). A linear probe predicting layer- $(\ell+1)$ coupling features from layer- ℓ features achieves $R^2 = 0.40$ on average, rising from 0.28 at early layers to 0.63 at late layers. Coupling dynamics become increasingly autonomous with depth: early layers’ coupling depends on token identity (which tokens are present), while late layers’ coupling is increasingly predictable from the prior coupling pattern alone (how tokens relate). This is consistent with the residual stream “converging” toward $\mathbf{M}$ ’s geometric attractor—but through the decoder, not through $\mathbf{M}^n$.

Test	Prediction	Observed	Verdict
T1: Closure	$\mathbf{M}^n$ error bounded	Diverges $2,561\times$ at $n=2$	NEGATIVE
T2: Spectrum	Graceful degradation	Both methods fail equally	MIXED
T3: Lattice	Closure > 0.5	Closure = 0.025 ($<$ random)	NEGATIVE
T4: M-power	Cosine > 0.5	Cosine = 0.10 ($13\times$ random)	MIXED
T5: Autonomy	$R^2 > 0.5$	$R^2 = 0.40$ (rising with depth)	MIXED+

11.8 The Honest Assessment: What $\mathbf{M}$ Actually Is

The diagnostic laboratory delivers a clear negative on the strict Koopman interpretation. $\mathbf{M}$ is **not** a Koopman operator: $\mathbf{M}^n$ does not capture n -step dynamics, the eigenvalues have no multiplicative closure, and the encoder- $\mathbf{M}$ -decoder pipeline is not $\mathbf{M}$ -invariant.

But $\mathbf{M}$ is also not just a shared weight. It has $13\times$ more structure than a random matrix of matched spectral norm (T4). Coupling dynamics become increasingly autonomous at depth (T5), consistent with the residual stream converging toward a geometry defined by $\mathbf{M}$. And the architecture sweep shows that $\mathbf{M}$ -based orbit memory composes at 99.6%, far better than state-based alternatives (58.8%).

The revised interpretation:

$\mathbf{M}$ is a **coupling geometry**—a landscape that the response heads navigate, not a dynamical operator that evolves the state. The semigroup property is approximate and mediated by the decoder: $T(s) \circ T(t) \approx T(s+t)$ holds because the response heads learn to compensate for $\mathbf{M}$ ’s invariance failures, not because $\mathbf{M}^{s+t} = \mathbf{M}^s \cdot \mathbf{M}^t$ captures the dynamics exactly. Composition works not because $\mathbf{M}$ is a perfect Koopman operator, but because $\mathbf{M}$ provides a *consistent geometric frame* that the decoders can align to.

This explains several observations:

- **Why composition fidelity is high** (99.6%): the same $\mathbf{M}$ at every layer means the decoders are all navigating the same geometry. Splitting a sequence into chunks preserves this consistency because $\mathbf{M}$ does not change.
- **Why the frozen- $\mathbf{M}$ test works** (1.01% gap): the geometry carries information because it defines the “directions of coupling.” Fresh decoders can learn to navigate a frozen geometry nearly as well as one they co-evolved with.
- **Why spectral crystallisation occurs**: prediction training discovers the dominant coupling directions. These become $\mathbf{M}$ ’s principal eigenvectors—not because $\mathbf{M}$ is a Koopman operator, but because eigenvectors are the natural basis for describing a bilinear coupling form.
- **Why RS-ORN predicts better but composes worse**: the state tensor adds information that $\mathbf{M}$ ’s geometry alone does not provide (accumulated context), but it breaks geometric consistency between chunks because the state is path-dependent.

Implications for architecture design. The Koopman framework remains useful as a *design heuristic* even though it fails as a literal description. The spectral structure of $\mathbf{M}$ genuinely matters (mode utilisation, crystallisation, rank scaling all have predictive power). But the architectural priority shifts from “make $\mathbf{M}$ a better Koopman operator” to “make the response heads compose better over a shared geometry.” Specifically:

- The CK loss on A, B is still valuable—not because it makes $\mathbf{M}$ Koopman-like, but because it regularises $\mathbf{M}$ ’s geometry to be self-consistent at multiple timescales, which helps the decoders compose.
- Orbit memory (OM-ORN) works because it queries $\mathbf{M}$ ’s geometry at multiple depths, giving the decoders a multi-scale view. The orbit points are not “Koopman trajectory points”—they are “geometry samples at different coupling strengths.”
- The delta-rule state update (Section 17) remains promising: it provides the prediction advantage of state while routing associations through $\mathbf{M}$ ’s geometry, maintaining geometric consistency.
- Spectral protection (preventing slow modes from collapsing) remains important, not for “reasoning modes” in the Koopman sense, but for maintaining a rich geometric landscape that the decoders can exploit.

What changes. We retire the claim that “ $\mathbf{M}$ IS a Koopman operator” in favour of “ $\mathbf{M}$ defines a coupling geometry with Koopman-like spectral properties.” The semigroup is approximate and decoder-mediated. The composition advantage of SharedM is real but comes from geometric consistency, not algebraic exactness. This is a weaker but more honest claim, and it directs future work toward the decoder–geometry interaction rather than the spectral properties of $\mathbf{M}$ alone.

12 Constructing a True Koopman Operator

The diagnostic laboratory shows that standard cross-entropy training does not produce a Koopman operator. But the question remains: *could* we build one? And if so, what would it take?

12.1 Why Standard Training Fails

The failure has a precise cause. Consider the ORN’s forward pass decomposed into three stages:

1. **Encoder** ϕ : embedding + positional encoding maps tokens into a d -dimensional space.
2. **Coupling** $\mathbf{M}$: the shared metric tensor computes attention coupling.
3. **Decoder** ψ_ℓ : per-layer response heads (W_V, W_O , FFN, corrections) transform the representation.

A true Koopman operator requires that ϕ ’s range is $\mathbf{M}$ -invariant: applying $\mathbf{M}$ to a vector in ϕ ’s range must produce another vector in ϕ ’s range. Formally: $\mathbf{M} \cdot \text{image}(\phi) \subseteq \text{image}(\phi)$.

Cross-entropy training has no incentive to enforce this. It optimises $\psi_L(\dots\psi_1(\mathbf{M}, \phi(x)))$ to predict the next token. The decoder ψ_ℓ at each layer compensates for wherever $\mathbf{M}$ sends the representation—it does not matter whether $\mathbf{M}$ ’s output is in ϕ ’s range, because ψ_ℓ rescues it. $\mathbf{M}$ is free to send representations *anywhere*, and the decoder maps them back. Over training, $\mathbf{M}$ and the decoders co-adapt into a pipeline that works for 1-step prediction but has no compositional structure.

This is exactly what T1 reveals: the 1-step prediction is excellent ($\text{MSE} \sim 10^{-5}$), but $\mathbf{M}^2$ is $2,561\times$ worse than sequential re-encoding. $\mathbf{M}$ moves the representation out of ϕ ’s range on the first application, and $\mathbf{M}^2$ applies $\mathbf{M}$ to an out-of-range vector, producing nonsense.

12.2 The TC0 Connection

Is this failure related to the TC0 barrier? The answer is nuanced.

$\mathbf{M}^n$ is a linear transformation. Linear transformations are computable in TC0 (matrix–vector multiplication). So even a *perfect* Koopman operator $\mathbf{M}$ cannot break TC0 through $\mathbf{M}^n$ alone. The expressiveness comes from the nonlinear encoder ϕ and decoder ψ : the encoder lifts the dynamics into a space where linear evolution is sufficient; the decoder maps back.

The TC0 barrier for SSMs applies to models with *diagonal* state transitions [21]. $\mathbf{M}$ is a full $d \times d$ matrix—it is not diagonal, and full matrix multiplication *can* represent permutations. RWKV-7 [23] breaks TC0 precisely by making state transitions non-diagonal and input-dependent. $\mathbf{M}$ is already non-diagonal. The obstacle is not representational capacity—it is *training*.

The precise claim: **the failure to learn a Koopman operator is a training problem, not an architecture problem.** The architecture (d -dimensional $\mathbf{M}$ with nonlinear encoder/decoder) has sufficient capacity. Standard cross-entropy training does not impose the right inductive bias (subspace invariance).

12.3 The Koopman Autoencoder Approach

The Koopman autoencoder [7, 8] provides the missing training signal. The architecture is:

$$z_t = \phi(x_t) \quad (\text{encoder: state} \rightarrow \text{latent}) \quad (6)$$

$$z_{t+n} \approx \mathbf{M}^n z_t \quad (\text{linear Koopman evolution}) \quad (7)$$

$$\hat{x}_{t+n} = \psi(z_{t+n}) \quad (\text{decoder: latent} \rightarrow \text{state}) \quad (8)$$

The training loss combines reconstruction, 1-step prediction, and **multi-step consistency**:

$$\mathcal{L} = \underbrace{\|x_t - \psi(\phi(x_t))\|^2}_{\text{autoencoder}} + \sum_{n=1}^N \underbrace{\|\phi(x_{t+n}) - \mathbf{M}^n \phi(x_t)\|^2}_{\text{Koopman invariance}} + \underbrace{\|x_{t+n} - \psi(\mathbf{M}^n \phi(x_t))\|^2}_{\text{multi-step prediction}} \quad (9)$$

The critical term is the **Koopman invariance loss**: $\|\phi(x_{t+n}) - \mathbf{M}^n \phi(x_t)\|^2$. This forces the encoder to produce representations that evolve linearly under $\mathbf{M}$. The encoder must find a subspace where $\mathbf{M}$ ’s linear dynamics approximate the true nonlinear dynamics—this is the Koopman eigenfunction discovery problem.

12.4 Adaptation to the ORN

Translating this to the ORN requires identifying what “ x_t ” and “ x_{t+n} ” mean. In a dynamical system, these are states at different times. In a language model, the natural analogue is: x_t is the residual stream at layer ℓ , and x_{t+n} is the residual stream at layer $\ell + n$. Each layer is one “time step” of the dynamics.

The adapted training loss:

$$\mathcal{L}_{\text{KO}} = \sum_{\ell=0}^{L-n} \|h^{(\ell+n)} - \mathbf{M}^n \cdot h^{(\ell)}\|^2 \quad (10)$$

where $h^{(\ell)}$ is the residual stream *after* layer ℓ ’s full processing (including response heads and FFN). This says: the residual stream at layer $\ell + n$ should be approximately $\mathbf{M}^n$ applied to the residual stream at layer ℓ .

There is a subtlety: the residual stream at layer ℓ is the *output* of ψ_ℓ , not the input. So we are asking $\mathbf{M}^n$ to predict the net effect of n layers of decoder intervention. This is a strong demand—it requires $\mathbf{M}$ to capture not just the coupling dynamics but also the response heads’ transformations.

A weaker but more tractable variant. Instead of requiring $h^{(\ell+n)} \approx \mathbf{M}^n h^{(\ell)}$ (which asks $\mathbf{M}$ to model the full pipeline), we can require that the *coupling pattern* is Koopman-linear:

$$\mathcal{L}_{\text{KO-coupling}} = \sum_{\ell=0}^{L-n} \|C^{(\ell+n)} - \mathbf{M}^n \cdot C^{(\ell)} \cdot (\mathbf{M}^n)^\top\|_F^2 \quad (11)$$

where $C^{(\ell)} = \text{softmax}(h^{(\ell)} \mathbf{M} h^{(\ell)\top} / \sqrt{d})$ is the coupling matrix at layer ℓ . This asks $\mathbf{M}^n$ to predict how the *pattern of attention* evolves across n layers, not how the representations themselves evolve. T5 shows this may be tractable: coupling autonomy increases with depth ($R^2 = 0.63$ at late layers), suggesting the coupling dynamics are more regular than the representation dynamics.

12.5 Radically Different Training: What Would It Look Like?

If we are willing to sacrifice standard language modelling performance for a true Koopman operator, several approaches become available:

Approach 1: Koopman-first, prediction-second. Train $\mathbf{M}$ and the encoder ϕ (embedding layer) using only the Koopman invariance loss for the first phase. The encoder learns to map tokens into a space where $\mathbf{M}$'s linear dynamics close. *Then* attach response heads and fine-tune for prediction with $\mathbf{M}$ frozen. This is the reverse of CORN's recipe (which does prediction first, composability second).

Risk: The Koopman-learned latent space may not be useful for prediction. The encoder discovers a mathematically closed subspace, but that subspace may not separate semantically meaningful distinctions. This is the T1 failure at the other extreme: perfect closure, useless representations.

Approach 2: Dictionary learning with known eigenfunctions. Instead of learning the encoder end-to-end, use a *fixed* dictionary of known Koopman eigenfunctions. For a d -dimensional $\mathbf{M}$ with eigendecomposition $\mathbf{M} = V\Lambda V^{-1}$, the eigenfunctions are projections onto V 's columns: $\psi_k(h) = v_k^\top h$. If we constrain the encoder to output representations as linear combinations of $\mathbf{M}$'s eigenvectors, then $\mathbf{M}$ -invariance is guaranteed by construction (eigenvectors are invariant under the linear map).

The architecture: decompose $\mathbf{M} = V\Lambda V^{-1}$, embed tokens as $\phi(x) = V\alpha(x)$ where $\alpha(x)$ is a learned coefficient vector. Then $\mathbf{M}\phi(x) = V\Lambda\alpha(x)$ —the evolution is just elementwise scaling of the coefficients by eigenvalues. This is *exactly* a structured state-space model in disguise: the “state” is α , the “transition” is Λ (diagonal), and V provides the change of basis.

Risk: This reduces to a diagonal SSM, which is bounded by TC0. The non-diagonal structure of $\mathbf{M}$ is lost. The full matrix power $\mathbf{M}^n = V\Lambda^n V^{-1}$ decomposes into eigenvalue powers, which is expressive for dynamics but not for composition.

Approach 3: Koopman autoencoder with periodic re-encoding. The key insight from T1: $\mathbf{M}^n$ fails because $\mathbf{M}$ pushes representations out of the encoder's range. The fix: *periodically re-encode* to bring representations back into range. Every k layers, apply: $h \leftarrow \phi(\psi(h))$ —decode to observation space, then re-encode to Koopman space.

The training loss becomes:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \sum_{\ell} \|h^{(\ell+k)} - \phi(\psi(\mathbf{M}^k h^{(\ell)}))\|^2 \quad (12)$$

This asks $\mathbf{M}^k$ to predict k steps of dynamics up to a re-encoding. The re-encoding corrects for drift out of the Koopman subspace. k controls the tradeoff: small k (frequent re-encoding) is

easy to train but $\mathbf{M}$ has little compositional work to do; large k (rare re-encoding) demands genuine Koopman closure but is harder to train.

This is architecturally identical to OM-ORN. The orbit memory computes $\mathbf{M}^1 h, \mathbf{M}^2 h, \dots, \mathbf{M}^K h$ and the cross-attention mechanism re-encodes by selecting relevant orbit points. If OM-ORN is trained with the Koopman invariance loss, it *becomes* a Koopman autoencoder with periodic re-encoding via cross-attention. This reframes OM-ORN not as “orbit memory” but as “Koopman autoencoder with attention-based re-encoding.”

Approach 4: Contrastive Koopman learning. DynCL [12] shows that contrastive learning can recover true system dynamics from observations, performing nonlinear system identification. The composition property *emerges* from learning correct dynamical structure. Applied to ORN: use InfoNCE loss on state pairs $(h^{(\ell)}, h^{(\ell+n)})$, trained to distinguish “same trajectory, n steps apart” from “different trajectories.”

The conjecture: a contrastive loss that respects temporal structure will implicitly discover Koopman eigenfunctions, because these are the features that evolve most predictably. Unlike the explicit Koopman invariance loss (which requires computing $\mathbf{M}^n$), the contrastive loss only requires pairs of representations—no matrix powers, no subspace invariance check. Composability would emerge as a side effect of correct dynamics, not as an imposed constraint.

Risk: The “emergent composability” claim from DynCL is demonstrated on simple physical systems, not language. It is unclear whether contrastive learning can discover Koopman eigenfunctions in a space as high-dimensional and unstructured as language representations.

12.6 The Fundamental Tension, Restated

The diagnostic laboratory reveals a sharp form of the prediction–composition tradeoff:

Good at prediction	Good at composition
Encoder/decoder co-adapt freely	Encoder’s range must be $\mathbf{M}$ -invariant
$\mathbf{M}$ sends representations wherever helpful	$\mathbf{M}$ must keep representations in-range
Per-layer decoders compensate	$\mathbf{M}^n$ must work without decoder help
1-step loss sufficient	Multi-step loss required
Result: standard ORN (val 0.46)	Result: Koopman ORN (unknown quality)

The TC0 barrier enters at the bottom row: a true Koopman $\mathbf{M}$ (linear evolution in the lifted space) is itself bounded by TC0. The nonlinear encoder/decoder can break TC0, but only by departing from Koopman linearity. **There may be no architecture that is simultaneously a true Koopman operator AND expressively beyond TC0.** The two desiderata are in formal tension: Koopman demands linear dynamics (TC0-bounded), expressiveness demands nonlinearity (TC0-breaking).

The resolution may be **approximate Koopman with periodic nonlinear correction**—essentially Approach 3 / OM-ORN. $\mathbf{M}$ evolves linearly for k steps (compositional, TC0-bounded), then a nonlinear re-encoding corrects the trajectory (expressive, TC0-breaking). The composability is approximate (k -step, not infinite-step) but may be sufficient for practical language modelling where the “composition depth” is bounded by the number of reasoning steps needed.

12.7 Proposed Experimental Programme

We propose four experiments to test whether a true (or approximately true) Koopman $\mathbf{M}$ can be trained for language:

1. **Koopman-ORN on TinyStories** ($d=128$, CPU). Train with the combined loss $\mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{KO}}$ from Equation 10. Sweep $\lambda \in \{0, 0.01, 0.1, 1.0\}$. Measure: (a) does $\mathbf{M}^n$ error decrease relative to standard ORN? (b) what is the prediction cost? (c) at what λ does T1 composition turn positive?
2. **Koopman-first recipe** ($d=128$, CPU). Phase 1: Koopman invariance loss only (no prediction). Phase 2: freeze $\mathbf{M}$, train response heads for prediction. Compare with CORN (prediction-first, composability-second). This tests whether the order matters.
3. **OM-ORN as Koopman autoencoder** ($d=128$, CPU). Add the Koopman invariance loss to OM-ORN’s training. The orbit memory already provides the re-encoding mechanism. Does the explicit Koopman loss improve T1/T4 scores while maintaining OM-ORN’s 99.6% composition?
4. **Contrastive Koopman** ($d=128$, CPU). Replace the explicit Koopman invariance loss with InfoNCE on $(h^{(\ell)}, h^{(\ell+n)})$ pairs. Does composability emerge without explicit enforcement? Compare eigenspectrum structure with the explicit Koopman variant.

All four experiments can run on CPU at toy scale, following the diagnostic laboratory’s principle of **cheap falsification before expensive optimisation**.

13 The Complex Depth Hypothesis: Prediction as Real Time, Reasoning as Imaginary Time

The connections between the ORN’s semigroup, diffusion models, and Deep-OSG converge on a single mathematical structure that unifies prediction and reasoning as two aspects of the same dynamics.

13.1 Time as a Continuous Input

Deep-OSG [9] revealed a critical design principle: the evolution time Δ must be an *input* to the network, not just the number of forward passes. Their architecture:

$$u_{\text{out}} = u_{\text{in}} + \Delta \cdot N_{\theta}(u_{\text{in}}, \Delta)$$

takes both the state u_{in} and the time step Δ as inputs. As $\Delta \rightarrow 0$, the output approaches the identity ($T(0) = I$, hard-coded by the multiplicative Δ). A single forward pass produces $T(\Delta)$ for any Δ .

The semigroup loss then compares $T(\Delta_1 + \Delta_2)$ against $T(\Delta_1) \circ T(\Delta_2)$ for randomly sampled Δ_1, Δ_2 . These are genuinely different computations because the network receives different Δ values—unlike our earlier implementation, which ran the same blocks multiple times and obtained trivially zero error.

Diffusion models have the same structure: the noise level $\sigma(t)$ is a continuous input to the denoiser. The consistency condition [5] ($f(x_t, t) = f(x_{t'}, t')$ for points on the same trajectory) is the semigroup property on the denoising process.

For the ORN, depth plays the role of time. Currently depth is discrete and implicit: the network does not know which “time” it is at. Making depth a continuous input parameter would unify the ORN with both frameworks.

13.2 The Wick Rotation: Real Time and Imaginary Time

In physics, a real process parameterised by time t can be analytically continued to complex time $t = \tau + i\beta$. This is the **Wick rotation**, which connects two apparently different theories:

- **Real time** τ : ordinary causal evolution. The propagator $e^{-i\hat{H}\tau}$ evolves quantum states forward in time. This is prediction: what happens next.
- **Imaginary time** $i\beta$: statistical mechanics. The partition function $Z(\beta) = \text{Tr}[e^{-\beta\hat{H}}]$ weights all states by their energy. This is integration over possibilities: what could happen, weighted by plausibility.

Same Hamiltonian $\hat{H}$. Same operator. Different direction in the complex plane. Real time gives dynamics (evolution). Imaginary time gives thermodynamics (exploration).

13.3 Complex Depth in the ORN

If $\mathbf{M}$ is the generator of the ORN’s semigroup, then $e^{t\mathbf{M}}$ evolves the residual stream. Extending t to the complex plane:

- **Real** t (prediction): $e^{t\mathbf{M}}$ amplifies the fast Koopman modes (large $|\lambda_k|$) and lets the slow modes decay. The representation converges toward the most likely continuation. This is the standard ORN: L layers of causal evolution.
- **Imaginary** $t = i\beta$ (reasoning): $e^{i\beta\mathbf{M}}$ *rotates* the state through $\mathbf{M}$ ’s eigenspaces without amplifying or damping. Every Koopman mode oscillates with frequency λ_k . The representation cycles through the space of possibilities without collapsing to any one.
- **Complex** $t = \tau + i\beta$ (mixed): simultaneous prediction (convergence along τ) and exploration (rotation along β). The balance between τ and β controls how much the network “thinks before deciding.”

The eigenvalue structure supports this. We observed that 89% of V1’s $\mathbf{M}$ eigenvalues are complex (with nonzero imaginary parts). These complex eigenvalues produce oscillatory dynamics even at real t —the residual stream already rotates as it evolves through depth. The real parts of the eigenvalues control convergence rate (prediction). The imaginary parts control oscillation frequency (exploration). Both are present in the same $\mathbf{M}$, learned by standard prediction training.

The prediction/reasoning duality is not an architectural addition—it is a property of the complex eigenvalue structure that $\mathbf{M}$ already possesses. Prediction training exploits the real parts. Reasoning would exploit the imaginary parts.

13.4 Practical Implementation: Depth as Continuous Input

The simplest way to introduce continuous depth without changing the ORN’s core architecture:

Level 0: Time-scaled coupling (minimal change). Scale $\mathbf{M}$ by a continuous depth parameter t :

$$q = h \cdot (t \cdot A), \quad k = h \cdot (t \cdot B)$$

At $t = 1$ this is the standard ORN. At $t < 1$ the coupling is weaker (shallower processing). At $t > 1$ the coupling is stronger (deeper processing). At complex $t = 1 + i\beta$, the coupling has both a convergent and a rotational component.

This requires *no new parameters*. The existing A, B are reused; only the scalar t changes. The response heads and corrections remain per-layer and unchanged.

Level 1: Deep-OSG adaptation (moderate change). Following Deep-OSG, make t an explicit input that multiplies the residual:

$$h_{\ell+1} = h_\ell + t_\ell \cdot f(\mathbf{M}, h_\ell)$$

where t_ℓ can be a learned per-layer time step or a globally scheduled parameter. The semigroup loss becomes non-trivial: $T(t_1+t_2)$ is a single forward pass with combined time, while $T(t_1) \circ T(t_2)$ is two forward passes with separate times.

Level 2: Complex-valued depth (ambitious). Allow t to have an imaginary component: $t_\ell = \tau_\ell + i\beta_\ell$. The imaginary part introduces oscillatory dynamics explicitly. This is mathematically equivalent to applying both the symmetric part of $\mathbf{M}$ (convergence, τ) and the antisymmetric part (rotation, β) with independent strengths.

We note that this is speculative and untested. The standard ORN with real $t = 1$ at every layer is the established baseline. Each level of extension should be validated independently before proceeding to the next.

13.5 The Unification

Framework	Generator	Time parameter	Semigroup property
Diffusion	Score function $\nabla \log p_t$	Noise level $\sigma(t)$	Consistency: $f(x_t) = f(x_{t'})$
Deep-OSG	Neural ODE f_θ	Evolution time Δ	$T(\Delta_1 + \Delta_2) = T(\Delta_1) \circ T(\Delta_2)$
World model	RSSM dynamics	Rollout steps k	Latent overshooting
ORN	Shared $\mathbf{M}$	Depth t (discrete $\rightarrow$ continuous)	CK on coupling geometry

All four frameworks learn a generator, parameterise it by a continuous time-like variable, and enforce or benefit from the semigroup property. The ORN’s contribution is identifying the generator as a *coupling geometry* ($\mathbf{M} = AB^T$) rather than a general dynamics function, and the Koopman interpretation that connects the eigenspectrum to the prediction/reasoning duality.

The complex depth hypothesis adds: real depth is causal prediction, imaginary depth is statistical exploration, and the balance between them is the “thinking speed” of the system. A model that operates at real t is fast and convergent (System 1). A model that adds imaginary β is slow and exploratory (System 2). Both use the same $\mathbf{M}$, the same response heads, the same corrections. The only change is the direction of traversal through $\mathbf{M}$ ’s geometry.

14 The CORN Training Recipe: From Prediction to Composable World Model

The theoretical programme of Sections 10–13 raises an immediate practical question: can we *train* the semigroup property into $\mathbf{M}$? This section documents what we learned — including the failures that shaped the final recipe.

14.1 The Bug That Revealed the Architecture

Our first attempt at a semigroup loss was the **Chapman–Kolmogorov (CK) identity**:

$$T(\Delta_1 + \Delta_2) \stackrel{?}{=} T(\Delta_1) \circ T(\Delta_2).$$

The implementation applied the same SharedM block multiple times. The loss was trivially zero from initialisation: applying identical blocks in sequence automatically satisfies $T(s) \circ T(t) = T(s+t)$ because there is no way for the network to distinguish “apply for time $s+t$ ” from “apply for time s , then for time t .” The test was tautological.

Wu & Chen’s Deep-OSG (2023) showed the fix: time must be a *continuous input* to the network, so that $T(\Delta_1 + \Delta_2)$ is a genuinely different computation from $T(\Delta_1) \circ T(\Delta_2)$. Our Level 0 implementation is minimal: scale the coupling matrices by a continuous time parameter t , so that

$$q = h \cdot (t \cdot A), \quad k = h \cdot (t \cdot B).$$

At $t = 1$ this is exactly the standard ORN — no new parameters, no architectural change. The semigroup constraint now has teeth: the network must learn an $\mathbf{M}$ whose geometry is *self-consistent under time composition*, not merely repeated application.

Verification was immediate. Semigroup error became non-trivial ($\text{MSE} \sim 0.034$, compared to the previous machine-zero). Gradients flowed cleanly to A and B . The $t = 1$ baseline was preserved exactly.

14.2 The Diagnostic Journey

With a working semigroup loss, we explored several training configurations. Each failure was informative.

CK-only training. Optimising the CK identity alone, with no prediction loss, produced an $\mathbf{M}$ that was perfectly composable — and perfectly useless. The model collapsed to a max-entropy coupling: every token attended uniformly to every other token. Information was destroyed. The semigroup property was trivially satisfied because $\mathbf{M}$ had degenerated to a scaled identity. *Lesson: composability without prediction is vacuous.*

CK on $\mathbf{M}$ + prediction on all parameters (from step 0). Adding cross-entropy (CE) prediction loss from the first step, with the CK loss routed only to A and B , produced a model that predicted well but was barely composable. The CE gradients dominated: they shaped $\mathbf{M}$ for greedy next-token prediction, and the CK loss was too weak to counteract this from the start. *Lesson: prediction pressure crushes composability if applied simultaneously from initialisation.*

CK on $\mathbf{M}$ + prediction on all (curriculum). A curriculum schedule — CK loss first, CE ramped in gradually — showed partial success. CK error dropped 15%. Entropy increased (the model maintained broader coupling patterns). But prediction cost 2.6% relative to the CE-only baseline. *Lesson: the curriculum helps, but the two objectives are in genuine tension.*

Time-scaled semigroup (the correct implementation). The Deep-OSG-inspired t -scaling was the breakthrough. With random (t_1, t_2) pairs drawn during training and the semigroup loss checking $T(t_1 + t_2) \approx T(t_1) \circ T(t_2)$:

- SG error dropped 33% relative to the curriculum variant.
- The second singular value σ_{20} reached 0.366 — three times the baseline, indicating genuine spectral structure rather than collapse.
- Prediction cost was 2.9% relative to CE-only.

The key finding. Prediction training *actively destroys* composability. During CE-only training phases, semigroup error rose by 57%. The semigroup loss reverses this trend, but only when time is a continuous input. This is not a failure of our implementation — it reflects a fundamental tension between greedy prediction and structural composability.

14.3 The Fundamental Tension Reframed

The 2.9% prediction cost is not a failure. It *is* the slow thinking.

A model trained only for next-token prediction converges its orbits rapidly — it commits to the most likely continuation and discards alternatives. This is fast, fluent, and shallow. A model with composable geometry maintains awareness of possibilities: higher entropy in the coupling, less convergent orbits, richer spectral structure in $\mathbf{M}$. This is slower, more exploratory, and deeper.

You cannot think fast and slow simultaneously. The prediction cost is a training-time investment in a richer world model — one where $\mathbf{M}$'s Koopman modes encode temporal regularities that pure prediction never discovers.

The analogy is precise: a chess engine that evaluates only the top move is fast but brittle. One that maintains a search tree over many continuations is slower per move but stronger per game. The semigroup loss builds the search tree into the geometry.

14.4 The Three-Phase Training Recipe (CORN)

The diagnostic journey converges on a three-phase recipe. One architecture, three training objectives, applied in sequence.

Phase 1: ORN (crystallise prediction geometry). Standard cross-entropy training at $t = 1$. $\mathbf{M}$ crystallises a prediction geometry with rank ~ 20 . This is the standard ORN training protocol and produces a model that matches GPT-2 Small quality at 91M parameters. No semigroup loss. The goal is simply to learn a good $\mathbf{M}$ for next-token prediction.

Phase 2: CORN (impose composability). Cross-entropy plus semigroup loss with t -scaling. At each step, draw random $(t_1, t_2) \in (0, 1]$ and enforce:

$$\mathcal{L}_{\text{SG}} = \|T(t_1 + t_2)[\mathbf{h}] - T(t_1)[T(t_2)[\mathbf{h}]]\|^2.$$

The semigroup loss is routed to A and B only — response heads (W_V , W_O , FFN) receive only CE gradients. The semigroup weight λ ramps linearly from 0 to a target value over the first 20% of Phase 2 steps. $\mathbf{M}$ becomes composable while response heads continue predicting.

Phase 3: ORN deploy (recover prediction). Freeze $\mathbf{M}$ (now composable). Fine-tune response heads with CE-only on the target domain. Because $\mathbf{M}$ is frozen, prediction pressure cannot destroy the composable structure. Response heads adapt *to* the composable $\mathbf{M}$ rather than reshaping it. Prediction quality is expected to recover, and the model retains composable geometry that prediction alone would never have discovered.

At inference: the standard ORN ($t = 1$, fast prediction) is used for fluent generation. When reasoning is needed — multi-step planning, coherence checking, self-correction — the model iterates at variable t , invoking the composable structure. The slow Koopman modes are present in $\mathbf{M}$ even during fast inference; they activate when the residual stream’s field state engages them.

14.5 What We Don’t Yet Know

The recipe is validated at toy scale (50M parameters, TinyStories). Several critical questions remain open:

1. **Downstream utility.** Does a composable $\mathbf{M}$ improve reasoning, planning, or long-range coherence on tasks that actually require temporal composition? We have spectral evidence but no downstream benchmark yet.
2. **Phase 3 recovery.** Does prediction quality fully recover when response heads are fine-tuned on frozen composable $\mathbf{M}$? A frozen- $\mathbf{M}$ training run on vast.ai is currently in progress and will provide the first answer.
3. **Scale dependence.** At V2 scale (125M+ parameters), does the prediction-composability tension persist, shrink, or reverse? Wider models have more spectral room; the tension may be an artefact of narrow d_{model} .
4. **Complex time.** Section 13 proposes $t \in \mathbb{C}$. Does adding an imaginary component $i\beta$ yield oscillatory modes that real- t training misses? The spectral theory predicts yes, but we have no empirical evidence.

5. **Recipe robustness.** Does the three-phase protocol work across datasets, model sizes, and hyperparameter choices, or does it require careful tuning of λ , curriculum schedule, and phase boundaries?

The CORN recipe is a hypothesis, not a conclusion. The diagnostic journey gives us confidence that the *direction* is correct — semigroup structure is trainable, time-scaling is the right mechanism, and the prediction cost is interpretable rather than pathological. Whether this translates to a practical advantage at scale is the next experiment.

Update: CORN stories at $d = 256$. The CORN stories experiment on TinyStories at $d_{\text{model}} = 256$ shows the prediction cost vanishes at sufficient scale. Val loss: ORN 2.182 vs CORN 2.183 (+0.06%). Rank compresses from 20 to 10 with zero quality cost. The fundamental tension between composability and prediction, measured at 2.9% at $d = 128$, is a small-scale artefact — at $d = 256$ there is enough spectral capacity for $\mathbf{M}$ to be both a good predictor and composable. This answers open question 3 (scale dependence): the tension shrinks with width and is negligible at $d \geq 256$.

15 The Semigroup IS the Diffusion Operator

The theoretical connections between the ORN’s semigroup, Koopman modes, consistency models, and diffusion have been developed across several sections. This section presents an empirical result that collapses these connections into a single identity: **the semigroup’s coupling strength t and the diffusion noise level σ are the same parameter.**

15.1 Background: Why This Matters

Standard diffusion models train a neural network to denoise images at varying noise levels. The noise level $\sigma(t)$ is an external schedule imposed on the network: a separate input, usually via sinusoidal positional embedding, that tells the network “this much noise was added, please remove it.” The network architecture does not change with σ —only the conditioning signal does.

The ORN’s t -scaled coupling ($q = h \cdot (tA)$, $k = h \cdot (tB)$) changes the *dynamics themselves*: at high t , $\mathbf{M}$ couples tokens more strongly; at low t , coupling is weaker. If these two parameters—the external noise level and the internal coupling strength—correspond, then the diffusion noise schedule is not an external imposition but an emergent property of $\mathbf{M}$ ’s spectral structure.

15.2 Experimental Setup

We trained five variants of the same ORN architecture ($d=96$, $L=6$, CIFAR-10 patches) as a denoiser within a standard DDPM framework (linear β schedule, 20 diffusion steps). Each variant maps the diffusion noise level σ to the semigroup coupling t via a different function $t = g(\sigma)$:

Mapping	Formula	Hypothesis
A: No link	$t = 1$ always	Baseline (standard ORN)
B: Stronger	$t = 1 + \sigma$	More noise $\rightarrow$ couple harder
C: Weaker	$t = 1/(1 + \sigma)$	More noise $\rightarrow$ couple less
D: Linear	$t = 0.5 + \sigma$	Linear ramp
E: Direct	$t = \sigma$	Coupling IS noise level

All other aspects—architecture, training objective (predict clean from noisy), noise schedule, number of steps—were identical across all five variants.

15.3 Results

Mapping	Mean MSE	vs baseline
E: $t = \sigma$ (direct)	0.0210	−2.3%
B: $t = 1 + \sigma$	0.0211	−1.9%
D: $t = 0.5 + \sigma$	0.0214	−0.5%
C: $t = 1/(1 + \sigma)$	0.0214	−0.5%
A: $t = 1$ (no link)	0.0215	—

Setting $t = \sigma$ gives the best denoising. The semigroup coupling strength and the diffusion noise level are, to a first approximation, the same number.

The advantage is concentrated at high noise levels (diffusion steps 15–19), where the model must recover coarse structure from heavily corrupted input. At low noise (steps 0–5), all mappings perform similarly because little denoising is needed. This is consistent with the spectral interpretation: at high t , $\mathbf{M}$ ’s fast Koopman modes (large eigenvalues) are amplified, providing the strong global coupling needed to find structure in noise. At low t , the slow modes contribute fine detail.

15.4 Connection to Field State Evolution

Section 4 showed that $\mathbf{M}$ ’s coupling subspace carries $1.8\text{--}3.4\times$ more per-token variance than random directions. The response heads steer signal into $\mathbf{M}$ ’s eigenspace. This is the spectral hierarchy that maps onto the noise schedule:

- $\mathbf{M}$ ’s top singular vectors (fast modes): handle the strongest signals, recovered first during denoising. Analogous to low-frequency structure in diffusion.
- $\mathbf{M}$ ’s lower singular vectors (slow modes): handle weaker signals, refined last. Analogous to high-frequency detail.
- The t -scaling amplifies or dampens this hierarchy: high t emphasises fast modes (coarse denoising), low t lets slow modes contribute (fine refinement).

The homogenisation observed in Section 4—tokens converging from cosine 0.21 to 0.91 through depth—IS denoising. Each layer propagates relational information through the field, reducing per-token noise toward consensus. The entropy oscillation (focus $\rightarrow$ diffuse $\rightarrow$ focus across layers) IS multi-scale refinement: alternating between global structure recovery and local detail recovery.

The ORN does not need a diffusion framework bolted onto it. Its natural forward dynamics—shared coupling applied to an evolving field state—already implement a spectral denoising process. The t -scaling makes this explicit: the coupling strength controls which modes are active, which is exactly what a noise schedule does in diffusion.

15.5 Implications

1. **No external noise schedule needed.** $\mathbf{M}$ ’s eigenspectrum IS the schedule. High t activates fast modes (coarse); low t activates slow modes (fine). One parameter controls both coupling and denoising.
2. **The semigroup property connects diffusion steps.** If $T(t_1 + t_2) = T(t_1) \circ T(t_2)$, then denoising from σ_1 to σ_3 via σ_2 is self-consistent. Standard diffusion trains each step independently; the semigroup guarantees compositional consistency.
3. **Prediction and diffusion use the same operator.** At $t=1$, the ORN predicts the next token (standard autoregressive mode). At $t=\sigma$ with varying σ , the same ORN denoises (diffusion mode). One architecture, one $\mathbf{M}$, two generation paradigms.
4. **The Koopman modes are the diffusion modes.** The eigenvalues of $\mathbf{M}$ control both the prediction timescales (Section 10) and the denoising frequencies. Fast eigenvalues:

rapid prediction, coarse denoising. Slow eigenvalues: gradual reasoning, fine denoising. Imaginary eigenvalues: oscillatory exploration (imagination).

15.6 The Complete Diffusion Process with Semigroup Coupling

For completeness, we write out the full training and generation procedures that connect the semigroup to diffusion. The key substitution is: wherever standard DDPM uses a timestep embedding $\text{emb}(t)$ as a conditioning signal, the ORN uses t as the coupling strength directly.

Forward process (noise addition, identical to DDPM). Given clean data x_0 , a linear schedule $\beta_1, \dots, \beta_T$, and $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$:

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

The noise standard deviation at step t is $\sigma_t = \sqrt{1 - \bar{\alpha}_t}$.

Training (denoising with $t = \sigma$).

1. Sample clean data x_0 from the dataset.
2. Sample diffusion step $t \sim \text{Uniform}(1, T)$.
3. Compute $\sigma_t = \sqrt{1 - \bar{\alpha}_t}$ and sample $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sigma_t \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$.
4. Set semigroup coupling $\tau = \sigma_t$ (the identity: coupling strength IS noise level).
5. Forward pass through ORN at coupling τ :

$$\begin{aligned} h &= \text{RMSNorm}(\text{Proj}(x_t)) + \text{PosEmb} \\ q^{(\ell)} &= h^{(\ell)} \cdot (\tau \cdot A), \quad k^{(\ell)} = h^{(\ell)} \cdot (\tau \cdot B) \quad (t\text{-scaled coupling}) \\ \hat{x}_0 &= \text{Head}(\text{Norm}(h^{(L)})) \end{aligned}$$

6. Loss: $\mathcal{L} = \|\hat{x}_0 - x_0\|^2$ (predict clean data).

Generation (DDPM reverse process with semigroup coupling).

1. Sample $x_T \sim \mathcal{N}(0, \mathbf{I})$ (pure noise).
2. For $t = T, T-1, \dots, 1$:
 - (a) Compute $\sigma_t = \sqrt{1 - \bar{\alpha}_t}$ and set coupling $\tau = \sigma_t$.
 - (b) Predict clean: $\hat{x}_0 = \text{ORN}(x_t, \tau)$.
 - (c) Predict noise: $\hat{\varepsilon} = (x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0) / \sigma_t$.
 - (d) DDPM update with stochasticity:

$$x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sigma_t} \hat{\varepsilon} \right) + \sqrt{\beta_t} z, \quad z \sim \mathcal{N}(0, \mathbf{I})$$

3. Return x_0 (the generated sample).

The $\sqrt{\beta_t} z$ term is the stochasticity that prevents collapse to the mean. Standard DDPM adds this fresh noise at every reverse step. In the ORN, this noise could alternatively be injected through the perturbative correction gates: $x = x + g \cdot (\text{correction}(x) + \sqrt{\beta_t} z)$, making the corrections serve double duty as both deterministic refinement and stochastic exploration.

Pseudocode.

```
# Training
for x0 in data:
    t = randint(1, T)
    sigma = sqrt(1 - alpha_bar[t])
```

```

    eps = randn_like(x0)
    x_t = sqrt(alpha_bar[t]) * x0 + sigma * eps
    x0_pred = model.denoise(x_t, coupling_t=sigma)
    loss = mse(x0_pred, x0)
    loss.backward(); optimizer.step()

# Generation
x = randn(shape) # pure noise
for t in range(T, 0, -1):
    sigma = sqrt(1 - alpha_bar[t])
    x0_pred = model.denoise(x, coupling_t=sigma)
    eps_pred = (x - sqrt(alpha_bar[t]) * x0_pred) / sigma
    x = (1/sqrt(alpha_bar[t])) * (x - beta[t]/sigma * eps_pred)
    if t > 1:
        x = x + sqrt(beta[t]) * randn_like(x) # stochasticity

```

What the semigroup adds. The semigroup property $T(\sigma_1) \circ T(\sigma_2) \approx T(\sigma_1 + \sigma_2)$ constrains the denoiser: denoising from $\sigma=0.8$ should be decomposable into denoising from $\sigma=0.5$ followed by $\sigma=0.3$. Standard DDPM makes no such guarantee—each step is trained independently. The CORN semigroup loss (Section 14) could be applied during diffusion training to enforce this, potentially improving multi-step consistency and allowing fewer reverse steps at generation time.

15.7 Note: The Path Integral Connection

The $t=\sigma$ identity closes a circle connecting four mathematical frameworks:

1. **M as Koopman operator** (Section 10): the eigenspectrum defines independent dynamical modes at different timescales.
2. **The semigroup as propagator:** $T(\sigma)$ propagates the field state through coupling space. The composition $T(\sigma_1) \circ T(\sigma_2) = T(\sigma_1 + \sigma_2)$ is the propagator composition rule from physics.
3. **The path integral:** the propagator is the sum over all paths, $K = \int e^{iS[\text{path}]}$, where the action S is defined by **M**. The G6 experiment (companion paper) demonstrated this concretely: a network trained with propagator composition computed exact Ising partition functions $Z = \int e^{-\beta H}$ to 3–4 significant figures.
4. **Diffusion as path sampling:** the DDPM reverse process samples one trajectory from the path integral. The stochasticity $\sqrt{\beta_t} z$ at each step explores different paths. The final sample x_0 is one draw from the distribution defined by integrating over all denoising trajectories.

These are four views of the same object: **M** generates the dynamics, the semigroup composes them, the path integral sums over all compositions, and diffusion samples from that sum. The $t=\sigma$ result says the coupling strength that defines the action IS the noise level that indexes the diffusion process.

What this means for training. The path integral perspective suggests three things:

First, the semigroup consistency loss (Section 14) is not an arbitrary regulariser—it enforces that the *propagator composes correctly*. In physics, a propagator that doesn’t compose produces inconsistent transition amplitudes. In diffusion, a denoiser whose steps don’t compose produces inconsistent multi-step denoising. The semigroup loss is the natural training signal for compositional generation.

Second, the G6 result (exact physics from propagator composition) suggests that CORN-trained models should be better at multi-step diffusion than standard models, because their denoising steps are guaranteed to compose. This could allow *fewer reverse steps* at generation

time: if $T(0.5) \circ T(0.5) = T(1.0)$ exactly, you can denoise in one large step instead of two small ones. Standard diffusion cannot make this guarantee.

Third, the β in $e^{-\beta H}$ (the inverse temperature) maps onto the coupling strength t . High temperature (β small, t small) = many states accessible = high entropy = exploration. Low temperature (β large, t large) = few states dominate = low entropy = convergence. This is the prediction/imagination duality (Section 13) expressed in the language of statistical mechanics. Training at multiple “temperatures” (multiple t values) is training the model to operate at all points on the entropy spectrum, from exploratory imagination to convergent prediction.

This result is at toy scale ($d=96$, 4K steps) and the effect is modest (-2.3%). At V2 scale with $\mathbf{M}$ ’s richer spectral structure (rank 20, condition number $> 10^9$), the spectral hierarchy is much more pronounced and the $t=\sigma$ correspondence should produce a larger advantage. This is a prediction, not a result.

16 Wild Conjectures

The following conjectures are speculative, untested, and possibly wrong. They are recorded because they connect ideas from across the programme in ways that could be productive even if the specific claims turn out to be incorrect.

16.1 The Noise Is the Entropy of Choice

Prediction is contractive. The standard ORN at $t=1$ takes all possibilities and collapses to the most likely next token. Every path not taken is discarded. The information about what *could* have happened is destroyed.

A composable $\mathbf{M}$ preserves paths. If $T(s+t) = T(s) \circ T(t)$, the intermediate state at step s contains information about all paths from 0 to s , not just the most likely one. The composition preserves the transition structure. A non-composable $\mathbf{M}$ (standard ORN) has no such guarantee: the intermediate states are optimised for the final prediction, not for representing the space of alternatives.

Conjecture 3 (Noise as entropy of choice). *The noise in diffusion is not corruption. It is the entropy of choice: the information about what could have happened, restored to a representation that prediction had discarded.*

The forward process (adding noise) restores the alternatives that prediction destroyed. At noise level σ , the state represents not “what happened” but “what could have happened at this scale of uncertainty.” The backward process (denoising) selects one path from the alternatives, reducing entropy step by step until a specific outcome crystallises.

This reframes the forward/backward process:

- **Forward** (prediction $\rightarrow$ noise): start from a specific outcome. Add noise = add back the choices that prediction discarded. At full noise, you have the full space of what could have been.
- **Backward** (noise $\rightarrow$ prediction): start from the full possibility space. At each step, make a choice (reduce noise = reduce entropy = select a path). The semigroup ensures these choices compose: choosing A then B is consistent with choosing $A+B$ directly.

16.2 Anisotropic Noise Along $\mathbf{M}$ ’s Eigenvectors

Standard diffusion adds isotropic noise: the same amount in all directions. But if $\mathbf{M}$ ’s eigen-spectrum defines the structure of the possibility space, the noise should be *anisotropic*: aligned with $\mathbf{M}$ ’s eigenvectors, scaled by their eigenvalues.

Conjecture 4 (Structured noise). *The natural noise for $\mathbf{M}$ -based diffusion is not $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$ but $\varepsilon \sim \mathcal{N}(0, \Sigma)$ where Σ is derived from $\mathbf{M}$ ’s spectral decomposition:*

$$\Sigma = V \text{diag}(|\lambda_1|, |\lambda_2|, \dots, |\lambda_d|) V^\top$$

where V and λ_i are $\mathbf{M}$ ’s eigenvectors and eigenvalues.

Fast modes (large $|\lambda_i|$) get more noise = more choice along those directions. Slow modes (small $|\lambda_i|$) get less noise = fewer choices. The eigenspectrum defines not just the dynamics but the shape of the possibility space.

The testable prediction: DDPM with anisotropic noise aligned to $\mathbf{M}$ ’s eigenvectors should produce better samples than DDPM with isotropic noise, because the noise matches the natural structure of the coupling geometry.

16.3 Diffusion as Diagnostic: “What Could Have Been?”

If the forward process restores discarded choices, it becomes a diagnostic tool. Given a prediction, run the forward process to see the *neighbourhood* in possibility space:

- σ small: “this word could have been a synonym” (local alternatives)
- σ medium: “this sentence could have gone differently” (structural alternatives)
- σ large: “this story could have been about something else” (thematic alternatives)

The noise levels correspond to $\mathbf{M}$ ’s eigenvalue scales. Fast modes vary first (big alternatives, different topics). Slow modes vary last (small alternatives, word choice). This gives a *multi-scale view* of the model’s uncertainty, richer than a single entropy number.

For planning: exploring the σ -neighbourhood of a proposed action shows the alternatives. Choosing the action with the best neighbourhood (most plausible alternatives, fewest catastrophic ones) is a form of robust decision-making. The composable semigroup ensures the neighbourhoods are consistent: the alternatives at scale σ_1 compose with the alternatives at scale σ_2 .

16.4 Why Composability Matters for All of This

Without composability, none of the above works. If $T(\sigma_1) \circ T(\sigma_2) \neq T(\sigma_1 + \sigma_2)$, then:

- The “paths” preserved by the semigroup are inconsistent. The alternatives at scale $\sigma_1 + \sigma_2$ don’t match the alternatives built from σ_1 and σ_2 .
- The noise-as-choice interpretation breaks down: adding noise and removing it doesn’t return to a consistent state.
- The diagnostic fails: the σ -neighbourhood of a prediction is different depending on how you got there.
- The diffusion process cannot be shortened: you cannot replace two small steps with one large step.

Composability is not an optional extra. It is the structural property that makes the possibility space *real*—not just a mathematical abstraction but a space the model can navigate, explore, and sample from. This is why CORN, despite its prediction cost, may be essential for any application that requires more than greedy next-token generation.

17 The State–Composition Duality and Hybrid Architectures

Our architecture sweep (Section 16, following experiments) revealed a stark tension: RS-ORN (residual state) achieves the best prediction (val 0.4253) but the worst composition fidelity (55% top-1 agreement), while OM-ORN (orbit memory) composes almost perfectly (99.78%) but predicts slightly worse. This is not a quirk of our architecture—it reflects a deep structural tradeoff. Recent theoretical and empirical work across the hybrid architecture literature illuminates both the cause and several promising resolutions.

17.1 The TC0 Barrier: Why State Breaks Composition

Merrill et al. [21] prove that state-space models with fixed-size state and diagonal (or scalar) state transitions are bounded by the circuit complexity class TC0—the same class as standard transformers. This means they *provably cannot* solve permutation composition, a canonical test of compositional reasoning. The key insight: lossy compression of context into a fixed-size state destroys exactly the relational information needed for composition.

Sanford et al. [22] sharpen this: solving L -level sequential function composition requires either $(L+1)$ full attention layers or 2^{3L^2} linear attention layers—an *exponential* gap. Full attention preserves all pairwise relationships (at quadratic cost); linear attention and SSMs compress these away.

Observation 6 (RS-ORN and TC0). *RS-ORN’s state accumulation rule $s_\ell = s_{\ell-1} + \alpha \cdot \text{compress}(h_\ell)$ is an additive compression—it projects the full residual stream into a fixed-size state via a linear map. This is precisely the class of operation bounded by TC0. The 55% composition fidelity is not a bug in our implementation; it is the theoretical ceiling for this class of state dynamics.*

OM-ORN escapes TC0 because orbit attention preserves full pairwise relationships between the query and the K orbit points. The orbit points are generated by $\mathbf{M}$ ’s dynamics, so the relational structure is grounded in $\mathbf{M}$ ’s geometry, but the attention mechanism itself is quadratic—hence composable.

17.2 Breaking Through: Non-Diagonal State Transitions

RWKV-7 (“Goose”) [23] demonstrates that non-diagonal, input-dependent state transitions provably surpass TC0. The state update uses a full matrix transformation:

$$S_{t+1} = f(x_t) S_t g(x_t)^\top + v_t k_t^\top \quad (13)$$

where f and g are input-dependent gating functions. The critical difference from scalar or diagonal decay: the full matrix multiplication $f \cdot S \cdot g^\top$ can represent permutation-like operations on the state, breaking through the TC0 expressiveness ceiling.

Conjecture 5 (M-routed state transitions). *If RS-ORN’s state transition is routed through $\mathbf{M}$ ’s geometry:*

$$S_{t+1} = f(\mathbf{M}, x_t) S_t g(\mathbf{M}, x_t)^\top + v_t (\mathbf{M} k_t)^\top \quad (14)$$

then the state dynamics inherit $\mathbf{M}$ ’s coupling structure while gaining the expressiveness needed for composition. The key associations are projected through $\mathbf{M}$ before storage, ensuring the state respects the shared coupling geometry.

This would give us the prediction advantage of RS-ORN (stateful, carries context) without the composition collapse (non-diagonal transitions preserve relational structure).

17.3 The Delta Rule: Surgical State Management

Yang et al. [24] introduce the Gated DeltaNet, which achieves both efficient state management and precise associative recall via the delta rule:

$$S_{t+1} = \alpha_t S_t - (S_t k_t) k_t^\top + v_t k_t^\top \quad (15)$$

The first term decays old associations (controlled by gate α_t). The second term *erases* the specific old value associated with key k_t . The third term writes the new value. Unlike uniform decay (which kills everything slowly), the delta rule surgically replaces specific associations.

Gated DeltaNet consistently beats Mamba-2, DeltaNet, and pure gating across language modelling, reasoning, retrieval, and length extrapolation [24].

Observation 7 (Delta rule and composability). *The delta rule separates “what to forget” from “what to remember.” Uniform decay (as in RetNet [26] or basic RS-ORN) forgets everything at the same rate, destroying the compositional structure embedded in relative associations. The delta rule preserves associations that haven’t been overwritten, maintaining the relational information that composition requires.*

For ORN, the natural adaptation routes key projections through $\mathbf{M}$:

$$S_{t+1} = \alpha_t S_t - (S_t \mathbf{M} k_t)(\mathbf{M} k_t)^\top + v_t (\mathbf{M} k_t)^\top \quad (16)$$

The state stores and retrieves associations in $\mathbf{M}$ ’s coupling space, ensuring consistency with the shared geometry.

17.4 The State Space Duality: $\mathbf{M}$ as Semiseparable Structure

Gu & Dao [25] establish that SSMs and attention are mathematically dual, connected through **semiseparable matrices**. An SSM with scalar-times-identity state matrix A is exactly equivalent to masked self-attention with a 1-semiseparable causal mask. The same sequence transformation has two algorithmic realisations: $O(T)$ recurrence or $O(T^2)$ attention.

The causal mask in the SSM view is $D_{ij} = \prod_{k=i+1}^j a_k$, encoding exponential decay between positions. In this framework, our SharedM implicitly defines a particular semiseparable structure for the token mixing transformation.

Conjecture 6 ($\mathbf{M}$ ’s eigenspectrum as decay rates). *If $\mathbf{M} = U\Sigma V^\top$ and we identify the singular values σ_i with decay constants $\gamma_i = \exp(-1/\sigma_i)$, then SharedM simultaneously defines:*

1. The **coupling geometry** (which tokens attend to which, via U and V)
2. The **temporal decay** (how quickly attention fades with distance, via Σ)

High-eigenvalue modes couple strongly and decay slowly (long memory). Low-eigenvalue modes couple weakly and decay rapidly (local attention). This gives multi-scale retention (cf. RetNet [26]) entirely from $\mathbf{M}$ ’s existing spectral structure—no additional per-layer parameters.

17.5 Hybrid Interleaving: The Jamba Principle

Jamba [27] interleaves attention and Mamba SSM layers at a 1:7 ratio. Ablations show no substantial quality difference between 1:3 and 1:7—a few attention layers suffice. The critical finding: pure Mamba fails to develop in-context learning; the attention layers develop induction heads that enable ICL. Early layers should be SSM-like (feature extraction); attention layers belong at strategic depths.

StripedHyena [28] confirms that interleaving heterogeneous operators outperforms uniform architectures at *all* compute budgets. The expressiveness hierarchy result of Sanford et al. [22] provides the theoretical backing: $(L+1)$ strategically placed full attention layers suffice for L -level composition.

Conjecture 7 (Striped ORN). *Most layers use SharedM (efficient coupling, $O(d^2)$ shared parameters). Every N th layer receives a lightweight per-layer correction to the coupling:*

$$\mathbf{M}_\ell = \mathbf{M} + \sum_{k=1}^K \alpha_{\ell,k} \Delta_k \quad (17)$$

where Δ_k are $K \approx 2\text{--}3$ shared “correction atoms” and $\alpha_{\ell,k}$ are per-layer scalar coefficients. This interpolates smoothly between fully shared (current ORN) and fully independent (standard transformer) at a cost of $\sim 3K$ additional parameters per layer. At critical composition depths, the correction atoms provide the expressiveness that SharedM alone cannot.

This is supported by the MASA framework [31], which decomposes attention projection matrices across layers into shared dictionary atoms, achieving 66.7% parameter reduction while maintaining performance. MASA’s finding that $\sim 4\text{--}8$ atoms suffice for hundreds of layers directly validates the memoisation hypothesis: coupling geometry is low-rank across layers, with only small per-layer corrections needed.

17.6 Griffin’s Split Gate: When to Remember vs When to Compose

Griffin [29] introduces the Real-Gated Linear Recurrent Unit (RG-LRU), interleaving gated linear recurrences with local attention at a 2:1 ratio. The recurrence gate interpolates between “update state” and “preserve state”—learning per-position when to accumulate context and when to maintain existing representations.

This maps directly to our perturbative correction gate. Currently, the ORN gate controls “how much correction to apply.” A richer design would split this into two gates:

- **State gate** g_s : controls how much state to accumulate (RS-ORN-like behaviour)
- **Orbit gate** g_o : controls how much to attend to $\mathbf{M}$ ’s dynamics (OM-ORN-like behaviour)

The model learns per-layer and per-position whether to operate in “state mode” (good for prediction, accumulating context) or “orbit mode” (good for composition, attending to dynamical structure). The full correction becomes:

$$h_\ell = h_{\text{orbital}} + g_s \cdot \text{state_update}(h, s) + g_o \cdot \text{orbit_attend}(h, \mathbf{M}) \quad (18)$$

This lets different layers specialise: early layers might favour state accumulation (building representations), while deep layers favour orbit dynamics (composing relationships).

17.7 The Koopman–SKOLR Connection: Spectral Modes as Structured Recurrences

SKOLR [30] establishes a formal equivalence between structured Koopman operators and linear RNN updates. By representing state as time-delayed observations, the Koopman operator’s spectral decomposition yields a structured linear recurrence where different eigenfunction components evolve at different rates.

This deepens our Koopman interpretation (Section 10). $\mathbf{M}$ ’s eigenvectors define Koopman eigenfunctions—directions along which the dynamics decouple. Each eigenfunction component can be tracked independently with its own memory horizon:

- Fast modes ($|\lambda_i|$ large): converge quickly, need short memory
- Slow modes ($|\lambda_i|$ small): evolve gradually, need long memory
- Oscillatory modes (λ_i complex): rotate at specific frequencies

For orbit memory design, this means the K orbit points need not be uniform steps through $\mathbf{M}$ ’s dynamics. Instead, each orbit point should track a *specific Koopman mode*, with mode-specific memory horizons determined by $\mathbf{M}$ ’s eigenspectrum.

17.8 Gauge Theory Interpretation: $\mathbf{M}$ as a Connection

Weiler et al. [32] formalise gauge equivariant networks as neural networks on fibre bundles, where the convolutional kernel must satisfy linear constraints to ensure equivariance under local gauge transformations. The *connection* in differential geometry prescribes how features transform when moved between points.

SharedM *is* a connection on the “representation bundle” over sequence positions:

- $\mathbf{M}$ defines how representations at different positions couple (parallel transport)
- The same $\mathbf{M}$ at every layer means a *flat* connection (zero curvature in the depth direction)
- Perturbative corrections introduce *curvature*—deviations from flat geometry
- Composition fidelity corresponds to *flatness*: a flat connection trivially composes

This suggests the perturbative corrections should be geometrically consistent with the connection. In gauge theory, curvature $F = dA + A \wedge A$. The analogous constraint: corrections should be second-order in $\mathbf{M}$:

$$\text{correction}_\ell \sim [\mathbf{M}, h_\ell] = \mathbf{M}h_\ell - h_\ell\mathbf{M} \quad (19)$$

rather than generic MLPs. This commutator form ensures the correction respects $\mathbf{M}$'s coupling structure and vanishes when h is already aligned with $\mathbf{M}$'s eigenvectors (i.e., when the orbital dynamics are already correct).

17.9 Synthesis: Proposed Architectural Modifications

Table 5 summarises the actionable modifications emerging from this analysis. The modifications are ordered by expected impact and implementation cost.

Table 5: Proposed architectural modifications to ORN, with source, expected effect, and parameter cost.

Modification	Source	Expected effect	Cost
Non-diagonal state via $\mathbf{M}$	RWKV-7 [23]	Break TC0, rescue composition	Moderate
Delta-rule through $\mathbf{M}$ geometry	DeltaNet [24]	Precise associations	Moderate
$\mathbf{M}$ -derived exponential decay	RetNet [26], SSD [25]	Multi-scale memory	~ 0
Striped: $\mathbf{M}$ + correction atoms	Jamba [27], MASA [31]	Composition at key depths	$\sim 3K$ per layer
Split gate (state + orbit)	Griffin [29]	Per-layer specialisation	Small
Eigenmode-specific memory	SKOLR [30]	Principled orbit design	Small
CK composition loss	Deep-OSG [9]	Regularise composability	Training only
Commutator corrections	Gauge theory [32]	Geometric consistency	~ 0

The key theoretical takeaway. The state/composition tradeoff is not a fundamental law—it is a consequence of *diagonal state transitions* and *uniform decay*. Recent work (RWKV-7, Gated DeltaNet, the expressiveness hierarchy) shows clear paths through it. For ORN specifically: keep SharedM as the coupling authority, but upgrade state dynamics to use $\mathbf{M}$'s full matrix structure (non-diagonal transitions, delta-rule updates, eigenmode-aware memory). This preserves the parameter efficiency thesis while addressing the composition failure.

The most promising single modification is the **M-routed delta rule**: state updates that erase and write associations in $\mathbf{M}$'s coupling space. This gives the prediction advantage of stateful models (RS-ORN) while preserving the relational structure that composition requires (OM-ORN). Combined with the split gate (letting the model learn when to use state vs orbit dynamics), this may resolve the duality entirely.

18 Reproducibility

All analyses can be reproduced from the SharedM-50M checkpoint:

```
cd experiments
python3 analyze_semigroup.py
```

Checkpoint: `results/c1_scaling/ckpt_sharedm_50M.pt` (191MB).
Plots are saved to `results/semigroup/`.

References

- [1] T. Arcari, A. Ferrara, and M. Gori. Entropy-Lens: Non-Monotonic Entropy Profiles Across Transformer Layers. *arXiv:2502.16570*, 2025.
- [2] C. Bodnar et al. Neural Sheaf Diffusion: A Topological Perspective on Heterophilic Graph Learning. *NeurIPS*, 2022.
- [3] M. Dehghani et al. Universal Transformers. *ICLR*, 2019.
- [4] Z. Lan et al. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. *ICLR*, 2020.
- [5] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. Consistency Models. *ICML*, 2023.
- [6] B. O. Koopman. Hamiltonian systems and transformation in Hilbert space. *PNAS*, 17(5):315–318, 1931.
- [7] B. Lusch, J. N. Kutz, and S. L. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9:4950, 2018.
- [8] Temporally consistent Koopman autoencoders for forecasting dynamical systems. *Nature Scientific Reports*, 2025.
- [9] T. Wu and Y. Chen. Deep-OSG: Deep Learning of Operators in Semigroup. *Journal of Computational Physics*, 2023.
- [10] N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. *arXiv:physics/0004057*, 1999.
- [11] Exact and Soft Successive Refinement of the Information Bottleneck. *Entropy*, 2023.
- [12] Self-supervised contrastive learning performs nonlinear system identification. *ICLR*, 2025.
- [13] Learning Hierarchical World Models with Adaptive Temporal Abstractions. *ICLR*, 2024.
- [14] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz. Modern Koopman Theory for Dynamical Systems. *SIAM Review*, 64(2):229–340, 2022.
- [15] S. L. Brunton, B. W. Brunton, J. L. Proctor, and J. N. Kutz. Koopman Invariant Subspaces and Finite Linear Representations of Nonlinear Dynamical Systems for Control. *PLOS ONE*, 11(2):e0150171, 2016.
- [16] J. J. Bramburger. On the Lattice Property of the Koopman Operator Spectrum. *arXiv:2507.23498*, 2025.
- [17] M. D. Kvalheim and S. Revzen. Existence and Uniqueness of Global Koopman Eigenfunctions for Stable Fixed Points and Periodic Orbits. *Physica D*, 425:132959, 2021.
- [18] M. Haseli and J. Cortés. Invariance Proximity: Closed-Form Error Bounds for Finite-Dimensional Koopman-Based Models. *arXiv:2311.13033*, 2023.
- [19] D. Conradie, N. Boullé, L. Loiseau, S. L. Brunton, and M. J. Colbrook. Trustworthy Koopman Operator Learning: Invariance Diagnostics and Error Bounds. *arXiv:2603.15091*, 2026.
- [20] Least-Squares Multi-Step Koopman Operator for Model Predictive Control. *arXiv:2601.11901*, 2025.

- [21] W. Merrill, J. Petty, and A. Sabharwal. The Illusion of State in State-Space Models. *arXiv:2404.08819*, 2024.
- [22] C. Sanford et al. A Provable Expressiveness Hierarchy in Hybrid Linear-Full Attention. *arXiv:2602.01763*, 2025.
- [23] B. Peng et al. RWKV-7 “Goose” with Expressive Dynamic State Evolution. *arXiv:2503.14456*, 2025.
- [24] S. Yang et al. Gated Delta Networks: Improving Mamba2 with Delta Rule. *ICLR*, 2025.
- [25] A. Gu and T. Dao. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. *ICML*, 2024.
- [26] Y. Sun et al. Retentive Network: A Successor to Transformer for Large Language Models. *arXiv:2307.08621*, 2023.
- [27] O. Lieber et al. Jamba: A Hybrid Transformer-Mamba Language Model. *arXiv:2403.19887*, 2024.
- [28] M. Poli et al. StripedHyena: Moving Beyond Transformers with Hybrid Signal Processing Models. *arXiv:2503.01868*, 2025.
- [29] S. De et al. Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models. *arXiv:2402.19427*, 2024.
- [30] A. Nath et al. SKOLR: Structured Koopman Operator Linear RNN. *ICML*, 2025.
- [31] H. Ji et al. MASA: Share Your Attention via Matrix-based Dictionary Learning. *arXiv:2508.04581*, 2025.
- [32] M. Weiler et al. Equivariant and Coordinate Independent CNNs: A Gauge Field Theory of Neural Networks. 2024.